UC Berkeley CS 185/285《Deep Reinforcement Learning》深度学习笔记

> 本笔记依据 UC Berkeley **CS 185/285（Deep Reinforcement Learning, Decision Making, and Control）**公开站点的 **Spring 2026** 版本整理：主页 https://rail.eecs.berkeley.edu/deeprlcourse/、Syllabus 页、25 讲讲义 `static/slides/lec-1.pdf` … `lec-25.pdf`、5 份作业 PDF `static/homeworks/hw1.pdf` … `hw5.pdf`，以及起始代码仓库 `berkeleydeeprlcourse/homework_spring2026`。讲义主讲 **Sergey Levine**。材料访问日期均为 **2026-11**。
>
> 定位：读者已有经典 RL 基础，并在同目录读过 CS234 强化学习学习笔记。故本笔记**不重复** MDP/贝尔曼推导，只补三件事：**（1）各算法的偏差-方差与收敛性分析；（2）工程实现细节与失败模式；（3）作业里真正要写的代码结构**。
>
> 标注：**【讲义】** = 可在对应讲次 PDF 中直接找到依据；**【补充】** = 讲义未展开但属该领域公认结果（如 Rainbow、BCQ、TD3、决策 Transformer、DPO），用于补齐知识地图，不冒充课程内容。涉及学期日期、分数、作业门槛均以当学期主页/PDF 为准。

1.1 定位：把 RL 当工程系统
1.2 先修与评分（公开信息）
1.3 讲次地图（25 讲）
1.4 与 CS234 的分工
2.1 动机
2.2 算法
2.3 偏差-方差：$O(\epsilon T^2)$ 从哪来
2.4 实现细节
2.5 失败模式
3.1 DAgger：用 on-policy 数据修补分布偏移
3.2 逆强化学习：学奖励而不是学动作
4.1 目标函数
4.2 解剖图：所有算法共用三步
4.3 三类代价
4.4 场景速查
4.5 三个常见误判
5.1 动机
5.2 推导
5.3 偏差-方差
5.4 实现细节
5.5 失败模式
6.1 动机
6.2 三级跳与 GAE
6.3 在线 AC 与 A3C
6.4 实现细节与失败模式
7.1 动机
7.2 从 DP 到 fitted Q-iteration
7.3 DQN：让 FQI 在 Atari 上跑起来
7.4 收敛性：为什么非线性会发散
7.5 实践清单
7.6 失败模式
8.1 六个补丁与 Rainbow
8.2 偏差-方差视角
8.3 失败模式
9.1 动机：连续动作下的 $\max$
9.2 DDPG
9.3 TD3 的三个补丁【补充】
9.4 SAC：最大熵 RL 的工程化身
9.5 对比与实现
9.6 失败模式
10.1 动机：on-policy 的浪费
10.2 重要性采样
10.3 PPO：两种约束
10.4 偏差-方差
10.5 实现细节（PPO 是细节地狱）
10.6 失败模式
11.1 两个 hack 何时合法
11.2 策略梯度 = 策略迭代

11.3 单调改进下界
11.4 自然梯度
11.5 TRPO vs PPO
11.6 实现与失败模式
12.1 为什么 RL 课要讲变分推断
12.2 潜变量模型与 ELBO
12.3 控制即推断
12.4 从 soft optimality 到 SAC
12.5 实现与失败模式
13.1 动机
13.2 朴素算法与致命问题
13.3 不确定性估计
13.4 规划：MPC 与 CEM
13.5 Dyna 与 MBPO：短 rollout + 自举
13.6 潜空间世界模型
13.7 偏差–方差：model-based 的独特之处
13.8 失败模式
14.1 动机
14.2 核心困难：反事实查询
14.3 三条技术路线
14.4 策略约束：SAC+BC 与 AWAC
14.5 IQL：完全不查 OOD 动作
14.6 CQL 与表达力：从高斯到 flow
14.7 离线到在线
14.8 失败模式
15.1 动机：为什么探索"理论上不可解、实践上必须做"
15.2 经典方法
15.3 伪计数与 RND
15.4 技能发现与互信息（Lec 23）
15.5 失败模式
16.1 先摆正预期
16.2 值迭代收敛
16.3 样本复杂度（无探索的简化设定）
16.4 FQI 的误差分解
16.5 理论到工程的翻译
17.1 动机：LLM 为什么需要 RL
17.2 LLM 生成作为 MDP
17.3 奖励模型：Bradley–Terry
17.4 PPO–RLHF 的工程细节
17.5 GRPO：去掉 critic
17.6 DPO：把 RL 变成分类【补充】
17.7 可验证奖励与推理 RL
17.8 失败模式
18.1 动机：从"一任务一策略"到通才
18.2 多任务 RL = 联合 MDP 上的单任务 RL
18.3 目标条件 RL 与 HER
18.4 后继特征
18.5 分层 RL
18.6 元学习（meta-RL）
18.7 失败模式
19.1 三个视角
19.2 现实世界的五个问题
19.3 与 LLM 的交汇
19.4 我的判断（明确标注为推断）
20.1 CS234：理论骨架 ↔ 工程细节
20.2 CS229：监督学习是 RL 的子程序
20.3 CS336：LLM 训练栈 ↔ RL 后训练
20.4 CS231n：视觉表示 ↔ 策略的输入
21.1 复习清单（按"能否不看笔记讲 3 分钟"自评）
21.2 自测题（20 题）
21.3 答案
22.1 课程与作业（一手来源）
22.2 教材
22.3 论文（按章节）
22.4 说明与未覆盖项

第 1 章 课程概览：这门课在教什么

1.1 定位：把 RL 当工程系统

CS 185/285 与 CS234 这类"经典 RL 理论"课的最大区别是：**它把 RL 当作需要工程化的学习系统**，而不是一组需要证明收敛的算子。Lec 1 的核心对比是（**【讲义】Lec 1**）：

维度	监督学习	强化学习
数据分布	i.i.d.，外部给定	**非 i.i.d.**：上一刻输出影响下一刻输入
监督信号	每样本有标准答案	只有标量奖励，常延迟、稀疏
优化对象	拟合 $p(y \mid x)$	最大化 $\mathbb{E}_{\tau\sim p_\theta}[\sum_t r(s_t,a_t)]$
典型失败	过拟合、OOD	分布偏移、方差爆炸、值发散、探索失败

Lec 1 的论断贯穿全课：**RL = 优化（optimization）**，深度学习 = 可扩展学习。二者相加，才使"数据驱动的 AI 能超越数据本身"——纯监督"不会尝试比数据做得更好"。这解释了为什么本课一半篇幅在讲"如何把 RL 目标优化稳"。

1.2 先修与评分（公开信息）

项目	内容
先修（Syllabus）	**CS189 或等价课程**；假设已熟悉 RL、数值优化、ML
先修（Lec 1）	CS182 / CS189 / CS289 / CS281A / CS282 或外校等价
工具	PyTorch 训练神经网络（自动微分）

Syllabus 的 Grading 一节给出四项，**权重和 100%**：

项目	权重	说明
编程作业	**50%**	5 次 × 10%，约两周一次，独立完成
期末项目	**20%**	1–3 人组；默认选题：Offline-to-Online RL、LLM RL
考试	**20%**	学期后段（Lec 1 称 week 12；Syllabus 称 4 月 15 日那周）
小测	**10%**	Lec 2 起每讲一个 Gradescope 小测，7 天内交，可重做取高分

其他规则：全部经 Gradescope 提交；**共 5 个 late days**，只可用于 5 次作业（不可用于小测/项目文档/互评）；Lec 1 称作业"应人人可满分"（评分重点是实现正确 + 报告给图给分析）；Lec 1 要求"不要用 AI 编码工具"。以上数字仅对 Spring 2026 版有效，**以课程当学期主页为准**。

1.3 讲次地图（25 讲）

标题取自主页 "Lecture Slides" 链接文本，与 PDF 首页一致：

讲次	主题	模块	讲次	主题	模块
1	Introduction	导论	14	LLM RL	LLM 后训练
2–3	Behavioral Cloning (I/II)	模仿	15–16	Model-Based RL (I/II)	模型方法
4	RL Basics	目标与解剖	17–18	Offline RL (I/II)	离线
5	Policy Gradients	REINFORCE	19	Exploration	探索
6	Actor Critic	baseline/GAE	20	RL Theory	理论
7	Value-Based RL	策略迭代→FQI	21–22	Midterm Review	复习
8	Q-learning in Practice	target net/DDPG	23	Advanced Exploration	技能发现
9	Advanced PG 1	IS/PPO	24	Multi-task RL	多任务/元学习
10	Advanced PG 2	TRPO/自然梯度	25	Challenges & Open Problems	开放问题
11–13	VI / VI in RL / Control as Inference	概率视角			

主线压缩成一行：

```
模仿(2-3) → 解剖(4) → 策略梯度(5-6) → 值方法(7-8) → 约束型PG(9-10) → 概率视角(11-13)
→ LLM RL(14) → 模型方法(15-16) → 离线(17-18) → 探索(19,23) → 理论(20) → 多任务(24) → 开放(25)
```

与作业一一对应：HW1 模仿、HW2 策略梯度、HW3 Q-learning+SAC、HW4 LLM RL、HW5 离线 RL。

1.4 与 CS234 的分工

主题	CS234 负责	本笔记负责
MDP / 贝尔曼	定义、压缩映射证明	只用结论
值迭代 / 策略迭代	表格收敛性、误差界	fitted Q-iteration 为何在函数逼近下发散
Q-learning	GLIE、Robbins-Monro 条件	DQN 三补丁：replay、target net、Huber
策略梯度	似然比、基线无偏、REINFORCE	GAE 的 λ 插值、PPO-clip 与 TRPO 的 KL 推导
探索	老虎机、UCB 遗憾界、PAC-MDP	伪计数、RND、技能发现
模仿	BC 的 $O(T^2\epsilon)$、DAgger 归约	多模态动作、扩散/flow、action chunking
RLHF	Bradley-Terry、DPO 推导	PPO-RLHF 工程细节、GRPO 组归一化与 KL 估计器
控制/搜索	MCTS、AlphaZero	MPC/CEM、Dyna、MBPO、潜空间世界模型

一句话：**CS234 回答"为什么理论上成立"，CS285 回答"为什么你的实现跑不起来、以及怎么让它跑起来"**。两份笔记记号已对齐（$\gamma, \lambda, V^\pi, Q^\pi, A^\pi$），可交叉查阅。配套实战文档见 CS285 项目上手速查。

第 2 章 行为克隆：把控制当成监督学习

2.1 动机

模仿学习（imitation learning）解决"**没有奖励、但有专家示范**"。Lec 2 的建模思路（**【讲义】Lec 2**）：把控制设成 ML 问题——数据是"专家在 o_t 下做了 a_t"，模型是 $\pi_\theta(a \mid o)$，损失是负对数似然。ALVINN（1989）与 NVIDIA 端到端驾驶（Bojarski 2016）说明：同一算法有时彻底失败、有时效果惊人。这个反差就是本章理论要解释的对象。

2.2 算法

$$\theta^\star = \arg\max_\theta \ \mathbb{E}_{(o,a)\sim\mathcal{D}} \left[\log \pi_\theta(a \mid o)\right].$$

```
D = collect_demonstrations()
for step in range(num_steps):
    batch = sample_batch(D, B)
    mu, log_std = policy(batch.obs)                        # 或 logits
    loss = -gaussian_log_prob(batch.act, mu, log_std).mean()
    # 离散动作: loss = cross_entropy(policy(batch.obs), batch.act)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

动作表示三选一：**离散**用 softmax + 交叉熵；**低维连续**用对角高斯 $(\mu_\theta, \sigma_\theta)$；**多模态/高维**用自回归离散化、混合密度、扩散/flow。部分可观测时，严格策略是 $\pi(a_t \mid o_{1:t})$，实践中用固定历史窗口或序列模型近似——讲义强调这是**建模选择**，不是环境性质。

2.3 偏差–方差：$O(\epsilon T^2)$ 从哪来

设单步犯错概率在**训练分布**上 $\le \epsilon$。最坏情况下，长度 T 的轨迹上期望犯错次数

$$\sum_{t=0}^{T} \mathbb{E}\left[\mathbb{1}\{\hat\pi(s_t) \neq \pi^\star(s_t)\}\right] \le \epsilon T + \epsilon T(T-1) = O(\epsilon T^2).$$

机制：**一旦犯错就进入训练分布之外的状态，那里错误率不再有 ϵ 保证（最坏到 1）**。第 1 步的 ϵ 会"传染"后续所有步——这就是**分布偏移（distributional shift）/ 复合误差（compounding error）**。Lec 2 的类比："复习数学却考古希腊文学"。

一个耐人寻味的推论：**数据里包含更多"错误 + 恢复"片段时，BC 反而可能更好**。NVIDIA 用左右偏置摄像头人为制造"偏离→打回"样本，正是把专家分布从一条窄轨迹扩成"管状区域"。

性质	监督学习	行为克隆
数据分布	固定 p_{data}	依赖策略 d_{π_θ}
泛化误差	$O(\epsilon)$	$O(\epsilon T^2)$ 最坏
增大数据	单调有帮助	有帮助，但**覆盖的状态分布**比数量更关键
失败表现	精度不够	一旦偏离就雪崩

2.4 实现细节

多模态是 BC 最经典的失败（【讲义】Lec 3）：人类示范在同一路口可左可右，对角高斯的均值落在两模态中间——"直行撞墙"。

自回归离散化：把 d 维动作每维离散成 k 个 bin，用序列模型逐维预测，复杂度 $O(dk)$ 而非 $O(k^d)$——把联合分布分解成链式条件概率。

Flow matching（HW1 第二部分）：设动作块 A、噪声 $A_0 \sim \mathcal{N}(0, I)$、流时间 $\tau \sim U(0,1)$，线性插值 $A_\tau = \tau A + (1-\tau)A_0$，回归目标速度

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}\left[\left\| v_\theta(o, A_\tau, \tau) - (A - A_0) \right\|_2^2\right], \qquad A_{\tau+1/n} = A_\tau + \tfrac{1}{n} v_\theta(o, A_\tau, \tau).$$

与扩散的关系：扩散可看作**层级化 VAE**（【讲义】Lec 12），flow matching 用直线插值路径替代复杂噪声调度，实现更简单、采样步更少。HW1 门槛：MSE 策略 reward ⩾ 0.5、flow matching ⩾ 0.7（**HW1 PDF**）。

Action chunking：一次预测 K 步动作块并**开环执行**，再重新查询。好处是降低决策频率、块内不累积误差、更易建模多模态；代价是块内无法反应突发状况。HW1 的 Push-T 正是此设定（5 维状态、2 维动作）。

2.5 失败模式

失败	现象	诊断	处理
分布偏移	loss 低但 rollout 崩	找第一个偏离帧	DAgger、数据增广、更大模型
多模态平均	动作落在模态中间	画动作直方图	混合密度、自回归、扩散/flow
因果混淆	依赖伪相关特征	消融输入通道	加历史、改采集；DAgger **不能彻底**解决
非马尔可夫	同观测对应不同动作	检查 o_t 是否含历史	序列模型 / 历史窗口
窄数据	加扰动即失效	测试时加噪声	预训练（broad）+ 后训练（narrow）

Lec 3 的"宽 vs 窄"总结极有实践价值：用**大量但质量一般**的数据预训练（broad），再用**少量高质量**数据后训练（narrow）。坏数据让你"见过各种情况"，好数据告诉你"用户要什么"；单用后者，机器人一犯错就懵。这与 LLM 的"预训练 + SFT"同构。

第 3 章 DAgger 与逆强化学习概览

3.1 DAgger：用 on-policy 数据修补分布偏移

$O(\epsilon T^2)$ 来自"策略访问分布 ≠ 训练分布"。最直接的修法：**让专家在策略自己访问到的状态上打标签**（Ross et al. 2011）。

```
D = 初始专家示范; pi = train_bc(D)
for i in 1..N:
    rollouts = run_policy(pi)                      # 1. 策略自己访问的状态
    labels   = query_expert(rollouts.states)        # 2. 专家标注动作
    D = D ∪ {(s, expert_label(s))}                  # 3. 聚合（不能丢旧数据）
    pi = train_bc(D)                                # 4. 重训
```

三个要点（**【讲义】Lec 2**）：**聚合而非替换**（否则遗忘）；专家标注**动作**而非轨迹，故需要**可交互专家**；常见变体是"专家+噪声"混合采集而非纯当前策略——讲义专门提问"这样还能保证修掉分布偏移吗"，答案是**不能严格保证**（聚合分布 ≠ 策略分布），但实践更安全。

理论：no-regret 归约给出总代价 $O(\epsilon_N T)$（对比 BC 的 $O(\epsilon T^2)$）。直觉：训练分布逐步逼近策略分布，单步错误不再被放大 T 倍。这是"改数据分布比改算法更有效"的经典案例。

项目	说明
主要成本	每轮人类标注（瓶颈）；可用 MPC/规划器当合成专家
稳定性	每轮重训易震荡；实践用"在线持续训练 + 新数据进 buffer"
安全	早期策略差，真机会撞；用混合策略或仿真
坑	专家标注与观测**时序对齐**差一帧；专家自身不一致

Spring 2026 的 HW1 **不要求**实现 DAgger（做的是 action chunking + flow matching，**HW1 PDF**），但 DAgger 是必考概念（Lec 2/21 都讲）。

3.2 逆强化学习：学奖励而不是学动作

为什么学奖励（**【讲义】Lec 13–14**）：模仿动作只复制"做了什么"，不推理后果；模仿意图（reward）能学到"什么结果好"，因此可用**完全不同的动作**达成同一目标。IRL 问题是**不适定**的：无穷多奖励能让专家最优（恒零奖励让一切策略最优）。

概率化（**【讲义】Lec 12–13**）：引入最优性变量 $\mathcal{O}_t$，$p(\mathcal{O}_t = 1 \mid s_t, a_t) = \exp(r_\psi(s_t, a_t))$，得

$$p(\tau \mid \psi) \propto \exp\left(\sum_t r_\psi(s_t, a_t)\right), \qquad Z(\psi) = \int \exp\left(\sum_t r_\psi(s_t, a_t)\right) d\tau.$$

Z 不可算，是 IRL 的核心难点。极大似然梯度：

$$\nabla_\psi \mathcal{L} = \mathbb{E}_{\tau \sim \pi^*}\left[\nabla_\psi r_\psi(\tau)\right] - \mathbb{E}_{\tau \sim p_\psi(\tau)}\left[\nabla_\psi r_\psi(\tau)\right],$$

第二项需从 $\exp(r_\psi)$ 诱导的分布采样——正好用一个策略 π 近似，得到交替算法（**【讲义】Lec 13**，Finn et al. 2016 *Guided Cost Learning*）：

```
repeat:
    samples = rollout(pi)                                          # 策略采样
    r = update_reward(demos, samples, importance_weights)    # 更新奖励（samples 当负样本）
    pi = rl_update(pi, r)                                          # 用奖励改进策略
```

IRL 即 GAN：若令 $D_\phi(s,a)=\dfrac{\exp(r_\psi)}{\exp(r_\psi)+\pi(a\mid s)}$，则 IRL 目标与 GAN 目标只差常数。**GAIL**（Ho & Ermon 2016）直接用普通判别器 + 策略梯度，不显式恢复奖励。讲义的 pros/cons：**+** 更简单、少活动部件；**−** 判别器收敛后"什么也没告诉你"，且**无法重新优化奖励**（换环境要重训）。

方法	核心思想	备注
MaxEnt IRL（Ziebart 2008）	最大熵选"最不偏"奖励	需解内层 MDP，贵
Guided Cost Learning（Finn 2016）	策略采样近似 Z，交替优化	深度 IRL 经典
GAIL（Ho & Ermon 2016）	判别器 + 策略梯度	大规模模仿主力
AIRL（Fu 2018）	奖励分解 $g(s)+h(s,a)$，利于迁移	恢复的奖励更可复用

问题	BC	DAgger	IRL
需要什么	固定示范	可交互专家	示范 + 模拟器
分布偏移	严重 $O(\epsilon T^2)$	缓解 $O(\epsilon T)$	不直接适用
能否超越专家	不能	不能	**能**
主要成本	一次标注	多轮标注	内层 RL + 对抗训练

> 【补充】IRL 与离线 RL 数学上高度重叠（都从固定数据推断"什么好"），区别在 IRL 额外假设"数据来自最优者"。

第 4 章 RL 目标与"算法解剖图"

4.1 目标函数

$$J(\theta)=\mathbb{E}_{\tau\sim p_\theta(\tau)}\left[\sum_{t=0}^{T-1}\gamma^t r(s_t,a_t)\right],\qquad p_\theta(\tau)=p(s_0)\prod_t \pi_\theta(a_t\mid s_t)p(s_{t+1}\mid s_t,a_t).$$

三种等价写法（**【讲义】Lec 4**）：

写法	表达式	用途
轨迹期望	$\mathbb{E}_{\tau\sim p_\theta}[r(\tau)]$	策略梯度出发点
状态–动作边缘	$\mathbb{E}_{(s,a)\sim d^\pi}\left[\frac{1}{1-\gamma}r\right]$	与平稳分布联系，理论分析

写法	表达式	用途
值函数	$\mathbb{E}_{s_0}[V^\pi(s_0)]$	值方法、AC 出发点

其中 $d^\pi(s,a) = (1-\gamma)\sum_t \gamma^t p(s_t = s)\pi(a \mid s)$。讲义提醒：**我们几乎从不直接优化平稳分布**，但"目标可写成线性代数"对读理论文章很有用。

4.2 解剖图：所有算法共用三步

【讲义】Lec 4：

```
1. 生成样本 (run the policy)      ← 真机最贵；仿真可 10000x 实时
2. 拟合模型/估计回报 (estimate)    ← Q、V、环境模型、或直接算 return
3. 改进策略 (improve policy)       ← 梯度上升、argmax、或用模型规划
       ↑_____________________________|  (回到 1)
```

算法族	第 2 步估什么	第 3 步怎么改进	代表
策略梯度	用样本估回报	对 J 梯度上升	REINFORCE、PPO
值方法	$Q^\star$ 或 $V^\star$	$\pi = \arg\max_a Q$，无显式策略	DQN、FQI
Actor–Critic	V^π / Q^π（当前策略）	critic 降压方差	A3C、SAC、PPO
Model-based	环境模型 $p(s' \mid s,a)$	规划 / 反传 / 造数据	MPC、Dyna、MBPO
进化/无梯度	不用	参数扰动	CEM、ES

4.3 三类代价

样本效率——唯一最重要的问题是"是否 off-policy"：model-based > off-policy Q > actor-critic > on-policy PG > 进化。但讲义强调：**样本效率低 ≠ 不能用**，墙钟时间与样本效率是两件事，仿真里 on-policy 常因"简单、稳、易并行"总耗时更短。

稳定性：值函数拟合在优化 Bellman 误差（不是回报），非线性逼近下常常不收敛；model-based 优化模型误差，但"模型更准"≠"策略更好"；**只有策略梯度是在对真实目标做梯度上升**。这解释了 PPO 为何比 DQN 省心。

假设：离散/连续、随机/确定、episodic/无限、易表示的是策略还是模型。

4.4 场景速查

场景	首选	理由
离散动作、仿真、样本便宜	PPO / A2C	稳、易调、易并行
离散动作、样本贵	DQN + Double + PER	off-policy，数据可复用
连续动作、样本贵	SAC / TD3	off-policy + 表达力

场景	首选	理由
有可微模拟器	模型反传 / MPC	样本效率天花板
只有离线数据	IQL / CQL / FQL	不需交互
LLM 对齐	PPO-RLHF / GRPO / DPO	见第 17 章

4.5 三个常见误判

1. **把 Q-learning 当梯度下降**：其 target 含 $\max$，是**不动点迭代**，故需 target network（见 7.3）。
2. **把"模型更准"当"策略更好"**：两者无单调关系。
3. **把 on-policy 的高方差当学习率问题**：调小 lr 只是"慢一点崩"。

第 5 章 策略梯度（REINFORCE）：似然比技巧

5.1 动机

值方法要先解 $\arg\max_a Q$，连续/高维动作下很麻烦；策略梯度**直接对参数化策略求导**，不需要 $\arg\max$，天然支持连续动作与随机策略。

5.2 推导

似然比（log-derivative）恒等式 $\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)$ 代入 $J = \int p_\theta r \, d\tau$：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\nabla_\theta \log p_\theta(\tau)\, r(\tau)\right], \qquad \log p_\theta(\tau) = \log p(s_0) + \sum_t \left[\log \pi_\theta(a_t \mid s_t) + \log p(s_{t+1} \mid s_t, a_t)\right].$$

环境项与 θ 无关，求导后消失：

$$\boxed{\nabla_\theta J = \mathbb{E}_{\tau\sim p_\theta}\left[\left(\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right)\left(\sum_{t'} \gamma^{t'} r(s_{t'}, a_{t'})\right)\right]}$$

Lec 5 的实现视角（**【讲义】Lec 5**）：与最大似然只差一行——把 `-logp.mean()` 换成 `-(logp * returns).mean()`，即"给每个样本乘上它有多好"。讲义原话：这不过是把**试错（trial and error）**形式化了。

```
for iteration in range(N):
    trajectories = [run_episode(pi_theta) for _ in range(batch_size)]
    returns = compute_returns(trajectories)              # 见 5.3
    loss = -(log_prob(pi_theta, obs, act) * returns).mean()
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

5.3 偏差–方差

估计器	公式	偏差	方差
全轨迹回报	$\sum_{t'=0}^{T-1}\gamma^{t'}r_{t'}$	无偏	最高
reward-to-go	$\sum_{t'=t}^{T-1}\gamma^{t'-t}r_{t'}$	无偏	较低
加 baseline	上式 $-b(s_t)$	无偏	更低

reward-to-go 为何无偏：$\mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)] = \nabla_\theta \int \pi_\theta da = \nabla_\theta 1 = 0$，故 t 之前的历史奖励可从权重中删去而不改变期望（"因果性"）。

baseline 为何无偏：对只依赖状态的 $b(s_t)$，

$$\mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) b(s_t)\right] = \mathbb{E}_{s_t}\left[b(s_t)\underbrace{\mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta]}_{=0}\right] = 0.$$

减 baseline 不改变期望但大幅降方差；最优 baseline 是 $V^\pi(s)$（引出第 6 章）。讲义实践建议："平均回报作 baseline 已相当不错"。

高方差的直觉（**【讲义】Lec 5** 的国际象棋例子）：赢了（$R=+1$），REINFORCE 给**整局所有动作**正权重，包括明显的坏棋；输了则给所有动作负权重，包括好棋。这些错误**在期望上抵消**（故无偏），但需要极多样本——这就是"高方差"的工程含义。量级上单样本梯度方差约 $\propto T\sigma^2\sigma_R^2$：轨迹越长、回报尺度越大，方差越大。

5.4 实现细节

细节	推荐	原因
batch size	比监督学习大 1–2 个数量级（几千 step 起）	梯度噪声大
学习率	3e-4 ~ 1e-3 + Adam	太大直接崩
回报归一化	除以 running std	稳定任务间量级
折扣	0.99；长程 0.995–0.999	太小会短视
梯度裁剪	`clip_grad_norm_(params, 0.5~1.0)`	防偶发爆炸
探索	策略本身随机（高斯 σ 别太小）	PG 的探索来自策略熵

5.5 失败模式

现象	根因	处理
回报完全不动	回报量级太大/太小	归一化回报；查 reward scale
熵迅速塌缩	过早锁定某动作	熵正则、增大 batch、减小 lr

现象	根因	处理
突然崩到 0	某次更新步子太大	梯度裁剪、PPO 裁剪
曲线锯齿严重	方差大 + 数据只用一次	增大 batch、GAE、换 AC
只在固定 seed 下 work	初始化/数据顺序敏感	多 seed 报均值±方差

第 6 章 Actor–Critic 与 GAE：把方差压下去

6.1 动机

REINFORCE 的无偏性靠整条 MC 回报，代价是方差。若能学 V^π 或 Q^π，就可用**自举（bootstrap）**换方差——即 actor–critic（AC）。

6.2 三级跳与 GAE

级别	优势估计 $\hat{A}_t$	偏差	方差
REINFORCE	$\sum_{t'=t}^{T}\gamma^{t'-t}r_{t'}$	无偏	最高
AC 单步 TD	$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$	有偏	最低
AC n–step	$\sum_{l=0}^{n-1}\gamma^l r_{t+l} + \gamma^n V_\phi(s_{t+n}) - V_\phi(s_t)$	中间	中间

Lec 6 的"天平"（**【讲义】Lec 6**）：critic 当 baseline = **+低方差 / −有偏**；MC 回报 = **+无偏 / −高方差**；"critic baseline + reward–to–go" = **+无偏 + 更低方差**。

GAE(λ)（Schulman et al. 2016）把 n 从 1 到 ∞ 全部加权：

$$\hat{A}_t^{\mathrm{GAE}} = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l} = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\hat{A}_t^{(n)}$$

λ	退化为	偏差	方差
0	单步 TD δ_t	最大（依赖 V_ϕ）	最小
1	MC 减 baseline	0	最大
0.95–0.98	实践甜点	中	中

讲义的说法是"**一次性在所有位置切开**（cut everywhere all at once）"：λ 把"在哪截断自举"做成平滑加权，效果类似折扣因子（$\gamma\lambda$ 是有效折扣）。

$O(T)$ 反向递推（HW2 必考）：$\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$。

```
adv = 0; advantages = zeros(T)
for t in reversed(range(T)):
    next_value = values[t+1] if t+1 < T else 0.0
    nonterminal = 1.0 - dones[t]                    # 关键：终止状态不 bootstrap
    delta = rewards[t] + gamma * next_value * nonterminal - values[t]
    adv = delta + gamma * lam * nonterminal * adv
    advantages[t] = adv
returns = advantages + values                       # 用于训练 critic
```

`nonterminal` 是**最高频 bug**：忘了它，critic 会在终止后"幻想"未来奖励，值函数整体偏高。

偏差的精确来源：① critic 误差（λ 越小越依赖自举，偏差越大）；② 有限 horizon 截断。注意：**减 baseline 只改变方差，不引入偏差**（常被误解）。

6.3 在线 AC 与 A3C

```
s = env.reset()
while not done:
    a ~ pi_theta(.|s); s2, r, done = env.step(a)
    delta = r + gamma * V_phi(s2) * (1-done) - V_phi(s)
    phi   -= lr_v  * grad(V_phi(s), delta**2)          # critic
    theta += lr_pi * grad(log pi_theta(a|s), delta)     # actor
    s = s2
```

单样本噪声极大且样本相关；A3C 用**多个并行 worker** 异步更新全局参数，以并行换 batch。**实践现状**：更常用同步向量化环境（64 env）+ PPO，更稳更易调试。

6.4 实现细节与失败模式

项目	推荐	说明
critic 结构	与 actor 共享 encoder 或独立	共享省算力但易互相干扰
critic 损失	MSE 或 Huber	Huber 对 outlier 更稳
critic 更新次数	每 actor step 1–2 次	太少不准，太多过拟合当前策略
优势归一化	batch 内 mean 0 / std 1	HW2 明确要求，且注明**有偏**（batch=1 时恒为 0）
值目标	`returns = adv + values`	别另算原始回报，保持一致

现象	根因	处理
critic loss 不降	值目标与输出尺度差太多	归一化回报；查 `(1-done)`
actor 学坏、critic 很准	critic 过拟合当前策略窄分布	增大探索、减小 critic lr
后期突然崩	熵塌缩 + 优势全负	熵正则、优势归一化、早停
换 env 表现差异大	回报尺度不同	按 env 归一化

现象	根因	处理
加了 GAE 反而更差	λ/γ 混淆或 done 处理错	先验证 $\lambda=0$ 与 $\lambda=1$ 两个极端

第 7 章 值函数方法：从策略迭代到 DQN

7.1 动机

AC 仍需策略网络。若动作离散且维度不高，可**只学 Q**，用 $\arg\max$ 隐式表示策略。优点（**【讲义】Lec 7**）：+ 可用于 off-policy 数据；+ 只有一个网络、无高方差策略梯度；− 非线性逼近下无收敛保证。

7.2 从 DP 到 fitted Q-iteration

策略迭代 = 评估 $Q^\pi(s,a)=\mathbb{E}[r+\gamma\mathbb{E}_{a'\sim\pi}Q^\pi(s',a')]$ + 改进 $\pi(s)=\arg\max_a Q^\pi$。值迭代把两步合并：$V_{k+1}(s)=\max_a[r+\gamma\mathbb{E}V_k(s')]$，$\mathcal{T}$ 是 γ-压缩，有唯一不动点。

FVI（用函数逼近替代表格）：

$$\phi\leftarrow\arg\min_\phi\tfrac{1}{2}\sum_s\left(V_\phi(s)-\max_a\left[r(s,a)+\gamma\mathbb{E}_{s'}[V_{\phi'}(s')]\right]\right)^2.$$

两个致命问题：**需要模型**才能对 a 取 max；**维度灾难**（枚举动作）。

FQI：把 max 挪进 Q，**不需要模型**（不真的执行 a'），且天然 off-policy：

$$\phi\leftarrow\arg\min_\phi\tfrac{1}{2}\sum_{(s,a)}\left(Q_\phi(s,a)-\left[r(s,a)+\gamma\max_{a'}Q_{\phi'}(s',a')\right]\right)^2.$$

代价：非线性逼近下无收敛保证（见 7.5）。

7.3 DQN：让 FQI 在 Atari 上跑起来

为什么原版会崩：target $y=r+\gamma\max_{a'}Q_{\phi'}(s',a')$ 依赖被优化参数，参数一动 target 也动——"追自己的尾巴"，回归问题不再有稳定解（**【讲义】Lec 8** 的 moving target 比喻）。

补丁	做法	作用
replay buffer	大 buffer + 随机采样	打破相关性、数据可复用，更像 FQI
target network	硬更新（每 C 步）或 Polyak $\phi'\leftarrow\tau\phi+(1-\tau)\phi'$（$\tau=0.005$）	让回归目标短期固定，成为良定义回归

$$L(\phi)=\mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}}\left[\left(Q_\phi(s,a)-\left(r+\gamma(1-d)\max_{a'}Q_{\phi'}(s',a')\right)\right)^2\right].$$

```
phi, phi_target = init(), init(); D = ReplayBuffer()
for step in range(total_steps):
    eps = schedule(step)                                  # 1.0 -> 0.05
    a = eps_greedy(Q_phi, s, eps); s2, r, done = env.step(a)
    D.add(s, a, r, s2, done); s = s2 if not done else env.reset()
    if step % update_every == 0:
        b = D.sample(B)
        with torch.no_grad():
            a_star = Q_phi(b.s2).argmax(1)                # Double DQN
            y = b.r + gamma * (1 - b.done) * Q_phi_t(b.s2)[arange, a_star]
        loss = F.smooth_l1_loss(Q_phi(b.s, b.a), y)       # Huber
        optimizer.zero_grad(); loss.backward()
        clip_grad_norm_(phi, 10.0); optimizer.step()
        if step % target_update == 0: phi_t.load_state_dict(phi.state_dict())
```

UTD（update-to-data ratio）（**【讲义】Lec 8**）= 每个环境 step 做几次梯度更新。online Q-learning、DQN、FQI 的差别本质是三条流水线的相对速度：UTD 高学得快但算力贵，太高会**过拟合**（离线 RL 里尤其明显）。

过估计与 Double Q：$\max$ 对噪声取上界，$\mathbb{E}[\max_a \hat{Q}] \geq \max_a \mathbb{E}[\hat{Q}]$，偏差经自举累积。修正（van Hasselt 2016）：

$$a^\star = \arg\max_{a'} Q_\phi(s', a'), \qquad y = r + \gamma(1-d) Q_{\phi'}(s', a^\star),$$

即**在线网络选动作、目标网络估值**，两者噪声源不同，过估计大幅缓解。更一般地，选择器可以是任何东西（包括 actor）：$y = r + \gamma(1-d) Q_{\phi'}(s', \pi_\psi(s'))$——这就是 DDPG/SAC 的 target，故**actor-critic 与 Q-learning 并没多大不同**。

7.4 收敛性：为什么非线性会发散

【讲义】Lec 8 的论证链：值迭代（精确备份）是 γ-压缩 → 收敛；**FVI 每步含"投影"**（把目标值投到函数类上），而**投影不是非扩张的** → "压缩 + 投影"不再保证收敛；在 ∞-范数下误差可能被放大。结论：**FQI 一般情况下不收敛，实践中也常不收敛**。工程含义：需要更多调参；target net 与 replay 特别重要（让"投影"接近精确备份）；不同 seed 差异大；Q 值会爆炸。

误差分解（预告第 16 章）：$\|\hat{Q} - Q^\star\|_\infty \lesssim \frac{\epsilon_{\text{approx}}}{1-\gamma} + \frac{\epsilon_{\text{sample}}}{(1-\gamma)^2}$，两个分母在 $\gamma \to 1$ 时都很"毒"。

7.5 实践清单

优先级	建议	原因
P0	先在 CartPole 验证实现	复杂任务上分不清"算法错"还是"调参错"
P0	replay buffer 越大越稳	更像 FQI，数据分布更平稳
P0	Double Q 默认开	几乎无副作用
P1	梯度裁剪 / Huber	Bellman 误差梯度可能很大
P1	ϵ 由高到低、lr 由高到低	先探索后精修

优先级	建议	原因
P1	n-step return	加速奖励传播，但有偏
P2	多 seed、保持耐心	DQN "very inconsistent between runs"；可能长时间不如随机

7.6 失败模式

现象	根因	诊断
Q 值单调涨到 10^6	过估计失控 / target 没冻结	打印 `q_max`，应 $\le r_{\max}/(1-\gamma)$
训练回报升但 eval 崩	过拟合 replay 的窄分布	看 train/eval 分离（HW3 要求解释）
完全不学	buffer 太小 / 更新太频	调大 buffer、降 UTD
CartPole 都学不会	忘 `(1-done)` 或 target 用了在线网	单元测试单步 target
方差大	单 seed	至少 3 seed

第 8 章 值函数专题（续）：Dueling、优先回放与 Rainbow

> 本章多为 **【补充】**：Spring 2026 讲义在 Lec 7–8 只讲 target network、double Q、UTD、连续动作与收敛性，**未系统讲** Dueling/PER/Rainbow。此处补齐知识地图，出处为原始论文。

8.1 六个补丁与 Rainbow

```
DQN ─┬─ Double DQN      (减过估计)   ← 讲义讲了
     ├─ Prioritized ER  (样本效率)
     ├─ Dueling         (表示能力)
     ├─ Multi-step      (奖励传播)
     ├─ Distributional  (学分布而非期望)
     └─ Noisy Nets      (探索)                    → Rainbow = 全部 + 统一超参
```

Dueling：$Q(s,a)=V_\eta(s)+A_\psi(s,a)-\frac{1}{|\mathcal{A}|}\sum_{a'}A_\psi(s,a')$。$V+A$ 分解**不可辨识**（给 V 加常数、A 减同一常数不变），减均值把 A 约束为零均值，从而固定 V 的语义（比减 max 更稳）。适用于"很多动作无关紧要"的任务；动作少时收益小。

PER：$p_i=|\delta_i|+\epsilon$，$P(i)\propto p_i^\alpha$，IS 权重 $w_i=(1/(N\cdot P(i)))^\beta$（$\beta$ 从 0.4 退火到 1）。实现用 sum-tree；新样本给 $p_{\max}$；优先级用 Double 的 TD 误差算。**坑**：忘 IS 权重 → 有偏；α 太大 → 反复学少量样本。

其他：n-step（$y_t=\sum_{l<n}\gamma^l r_{t+l}+\gamma^n\max_a Q_{\phi'}(s_{t+n},a)$，$n=3$ 常用）；C51 学 $Z(s,a)$ 的 51 个 atom（交叉熵）；NoisyNet 用参数噪声替代 ϵ-greedy。

Rainbow 消融（Hessel et al. 2018）：移除 PER 掉分最多，其次 multi-step，再是 distributional/double/noisy，Dueling 最少。**教训：**"**数据怎么选"比"网络怎么设计"更重要**——这也预示了后来 offline RL 研究集中在"采样/保守性"。

8.2 偏差–方差视角

这些补丁可以按"改的是偏差还是方差"重新归类，这比记名字更有用：

补丁	主要作用	机制
Double Q	降**偏差**	断开"选动作"与"估值"的同一噪声源，抑制 $\max$ 的上偏
Dueling	降**方差**	共享 $V(s)$，让"动作无关"的部分用所有样本一起估
PER	降**方差**（梯度估计）	把算力集中在高 TD 误差样本；但若丢 IS 权重则引入**偏差**
n-step	降**偏差**（更快传播奖励），升**方差**	用更多真实奖励替代自举
Distributional	降**方差**（对回报的估计）+ 提供风险信息	学完整分布而非只用均值
NoisyNet	不改偏差/方差，改**探索**	状态相关的参数噪声

一句话：Double 修上偏、Dueling 省样本、PER 换效率（有偏风险）、n-step 换传播速度、Distributional 换信息量。**任何"降方差"的手段几乎都以"升偏差"或"多算力"为代价**——这正是第 16 章误差分解的工程版本。

8.3 失败模式

现象	根因	处理
Q 值尺度失控	过估计 + 自举	Double、target net、Huber、降 lr
优先回放过拟合	α 太大、buffer 太小	调小 α、增大 buffer
Dueling 反而更差	动作少、V/A 分解不稳	换普通 head 或加 A 的 L2
n-step 引入偏差	n 太大	$n=3$，检查终止处理

第 9 章 连续动作：DDPG、TD3、SAC

9.1 动机：连续动作下的 $\max$

【讲义】Lec 8 给三条路：

方案	做法	代价
梯度优化	内层对 a 做 SGD 最大化 Q	慢

方案	做法	代价
随机优化	采样 N 个动作取最大（sample-and-rank / CEM）	不精确，但简单可并行
学近似 maximizer	训练 $\mu_\theta(s) \approx \arg\max_a Q$	引入 actor 偏差，但最快

第三条即 **DDPG**（Lillicrap et al. 2016）。讲义点破本质：**actor 只是"近似 argmax"的可学习模块**。

9.2 DDPG

$$J(\theta) = \mathbb{E}_{s\sim\mathcal{D}}\left[Q_\phi(s, \mu_\theta(s))\right], \qquad \nabla_\theta J = \mathbb{E}\left[\nabla_a Q_\phi(s,a)\big|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)\right].$$

```
a = mu(s) + noise; D.add(...)                        # 探索噪声（OU 或高斯）
a2 = mu_t(b.s2); y = b.r + gamma*(1-b.d)*Q_t(b.s2, a2)
L_Q  = mse(Q(b.s, b.a), y)                           # critic
L_pi = -Q(b.s, mu(b.s)).mean()                       # actor
polyak(mu_t, mu, 0.005); polyak(Q_t, Q, 0.005)
```

偏差-方差：target 用 $Q_t(s', \mu_t(s'))$（旧 actor，非 $\arg\max$）→ **有偏（低估）**；Q 自身过估计 → 方向相反、大致抵消但不保证。确定性策略梯度只对 s 求期望，方差低于随机 PG，代价是**探索必须外挂噪声**。

9.3 TD3 的三个补丁【补充】

补丁	做法	作用
Clipped double-Q	$y = r + \gamma(1-d)\min_i Q_{\phi_i'}(s', \tilde{a}')$	抑制过估计
Target policy smoothing	$\tilde{a}' = \mu_{\theta'}(s') + \mathrm{clip}(\epsilon, -c, c)$	防 actor 利用 Q 的尖峰
Delayed policy update	每 $d=2$ 次 critic 更新才更新一次 actor	让 critic 先收敛

经验：**clipped double-Q 贡献最大**；target smoothing 在易过拟合 Q 的任务上关键。

9.4 SAC：最大熵 RL 的工程化身

$$J(\pi) = \sum_t \mathbb{E}\left[r(s_t,a_t) + \alpha\mathcal{H}(\pi(\cdot\mid s_t))\right].$$

熵的三个作用（**【讲义】Lec 13**）：探索（不必外挂噪声）、多峰最优时"保留选择"、与"控制即推断"一致（第 12 章）。软备份：

$$y = r + \gamma(1-d)\left[\min_i Q_{\phi_i'}(s', a') - \alpha \log \pi(a'\mid s')\right], \quad a' \sim \pi(\cdot\mid s').$$

三个损失：

$$
\begin{aligned}
L_Q &= \sum_i \mathbb{E}\left[(Q_{\phi_i}(s,a) - y)^2\right], \\
L_\pi &= \mathbb{E}_s\left[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)\right], \quad a = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon), \\
L_\alpha &= \mathbb{E}\left[-\alpha\left(\log \pi_\theta(a \mid s) + \bar{\mathcal{H}}\right)\right], \quad \bar{\mathcal{H}} = -|\mathcal{A}|.
\end{aligned}
$$

温度用**对偶梯度下降**求（把"熵 ≥ 目标"当约束，α 是拉格朗日乘子；HW3 PDF 3.5 节有完整推导）。

重参数化：$\nabla_\theta \mathbb{E}_{a\sim\pi_\theta}[Q] = \mathbb{E}_\epsilon[\nabla_\theta Q(s, \mu_\theta + \sigma_\theta \epsilon)]$——方差远低于 score-function，单样本可用；**只适用连续潜变量**，PyTorch 要用 `rsample()`。

tanh squash 的 log-det 修正（高频 bug）：

$$
\log \pi(a \mid s) = \log p(u \mid s) - \sum_i \log\left(1 - \tanh(u_i)^2\right), \quad a = \tanh(u).
$$

Q 值里也必须用 squash 后的动作，两处不一致会让自动温度跑偏。

9.5 对比与实现

维度	DDPG	TD3	SAC
策略	确定性	确定性	**随机高斯**
探索	外挂噪声	外挂噪声	**熵驱动**
过估计	严重	clipped double-Q	clipped double-Q + 熵
超参敏感度	高	中	**低**（自动温度）
定位	早期基线	连续控制基线	**默认首选**

项目	推荐	说明
网络宽度	256×2 MLP	MuJoCo 标准
双 Q	必须	否则过估计
target τ	0.005	太小慢、太大不稳
batch / buffer	256 / 10^6	off-policy 不需大 batch，但 buffer 不能小
动作缩放	按 env action bound	忘了会学不会
奖励缩放	除以 running std	MuJoCo 奖励量级差异大
评估	用均值动作	降低评估方差

9.6 失败模式

现象	根因	处理
Q 爆炸	双 Q 没生效 / 忘 `(1-d)`	打印 q_min/q_max，量级应在 $[-\frac{1}{1-\gamma}, 0]$ 附近

现象	根因	处理
动作饱和在 ±1	tanh 饱和 + 梯度消失	减小输出层初始化、加 entropy
学不会走路	奖励尺度 / 终止信号错	先跑 InvertedPendulum sanity（HW3 要求 return≈1000）
后期退化	replay 太旧 + 过拟合	增大 buffer、target smoothing
多峰任务只学一个模态	单峰高斯	换扩散/flow 策略（见 14.6）

第 10 章 重要性采样与 PPO：一次数据多步更新

10.1 动机：on-policy 的浪费

策略梯度每更新一次就得丢弃这批数据（π_θ 变了）。但采样最贵（真机 1x 实时）。我们想要：**用 π_{old} 采一批，然后对 θ 做多步更新**。障碍是 $J(\theta)=\mathbb{E}_{\tau\sim p_\theta}[r(\tau)]$ 的期望分布依赖 θ。

10.2 重要性采样

基本恒等式 $\mathbb{E}_{x\sim p}[f]=\mathbb{E}_{x\sim q}\left[\frac{p}{q}f\right]$，用在状态–动作边缘上（【讲义】Lec 9）：

$$\nabla_\theta J=\mathbb{E}_{(s,a)\sim d^{\pi_{\text{old}}}}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\text{old}}(a\mid s)}\nabla_\theta\log\pi_\theta(a\mid s)A^{\pi_{\text{old}}}(s,a)\right].$$

（严格推导还含状态分布之比 $\frac{d^{\pi_\theta}}{d^{\pi_{\text{old}}}}$，这是"两个 hack"中的第二个，第 11 章处理。）

方差问题：IS 估计方差随 $\rho=\pi_\theta/\pi_{\text{old}}$ 偏离**指数级**恶化——少数样本权重巨大，把估计拖飞。故必须限制 ρ。

10.3 PPO：两种约束

裁剪版（工程主流）：

$$L^{\text{CLIP}}(\theta)=\mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta),1-\epsilon,1+\epsilon)\hat{A}_t\right)\right].$$

情形	$\hat{A}>0$（好动作）	$\hat{A}<0$（坏动作）
ρ 在区间内	正常上升	正常下降
ρ 超上界	**截断**（不再奖励"过度提高概率"）	继续下降（不阻止降坏动作）
ρ 低于下界	继续上升（不阻止提好动作）	**截断**

直观：**裁剪只在"已朝正确方向走太远"时生效**，从而给多步更新一个安全边界。ϵ 常取 0.1–0.2（HW4 用 0.2）。

KL 惩罚版：$L=L^{\text{PG}}-\beta\mathbb{E}_t[\text{KL}(\pi_{\theta_{\text{old}}}\|\pi_\theta)]$，$\beta$ 可按 KL 自适应（太大加倍、太小减半）。讲义评价：**clip 是 KL 版的一个启发式近似**，实践中更好调。

```
for iteration in range(N):
    batch = [run_episode(pi_old) for _ in range(num_envs)]      # 固定 pi_old
    obs, act, logp_old, rew, done, val = flatten(batch)
    adv = gae(rew, val, done, gamma, lam); ret = adv + val
    adv = (adv - adv.mean()) / (adv.std() + 1e-8)
    for epoch in range(K):                                      # K = 3~10
        for mb in minibatches(obs, act, logp_old, adv, ret):
            logp  = log_prob(pi, mb.obs, mb.act)
            ratio = torch.exp(logp - mb.logp_old)
            policy_loss = -torch.min(ratio*mb.adv,
                          torch.clamp(ratio, 1-eps, 1+eps)*mb.adv).mean()
            loss = policy_loss + vf_coef*mse(V(mb.obs), mb.ret) \
                   - ent_coef*pi.entropy().mean()
            loss.backward(); clip_grad_norm_(params, 0.5); step()
    pi_old.load_state_dict(pi.state_dict())
```

10.4 偏差–方差

组件	偏差	方差	备注
IS 比值（未截断）	无偏	**可能无穷**	长序列指数恶化
PPO-Clip	**有偏**（故意）	受控	偏差来自"截断后不再优化"
PPO-KL	有偏（罚项改变最优解）	受控	β 决定偏差大小
GAE(λ)	随 $\lambda\downarrow$ 增大	随 $\lambda\uparrow$ 增大	见 6.2
多 epoch 复用	随 epoch 增大	—	数据越"旧"越偏

核心权衡：多 epoch 本质是拿"偏差"换"样本效率"。K 太大（如 20）会让策略跑出 trust region，性能崩。

10.5 实现细节（PPO 是细节地狱）

细节	推荐	为什么重要
并行 env	8–64	稳定 batch 统计量
rollout 长度	每 env 128–2048 step	太短 GAE 偏差大
epochs / minibatch	3–10 / 64–256	太大崩、太小噪声大
clip ϵ / λ / γ	0.1–0.2 / 0.95 / 0.99	标准配置
熵系数	0.0–0.01	太大不收敛、太小早熟
值损失系数 / 梯度裁剪	0.5 / 0.5	标准
优势归一化	必做	任务间量级差异
观测归一化	必做（running mean/std）	视觉/物理差异巨大
奖励缩放	running std 或 clip $[-10, 10]$	最常被忽略

细节	推荐	为什么重要
输出层初始化	正交 + gain 0.01	初始策略接近均匀

10.6 失败模式

现象	根因	诊断
熵迅速到 0、回报卡住	熵系数太小 / lr 太大	看 entropy 曲线
先升后崩	clip 太大或 epoch 太多	减 K、减 ϵ
完全不动	奖励没归一化 / 优势全 0	打印 adv 统计量
值 loss 爆炸	回报尺度大	归一化 return、clip value loss
seed 间差异大	并行 env 太少	增加 env 数

第 11 章 TRPO 与自然梯度：把"不要走太远"写成约束

11.1 两个 hack 何时合法

第 10 章的两个近似是：① 用 π_{old} 的状态分布代替 π_θ 的；② 用 $A^{\pi_{\text{old}}}$ 代替 A^{π_θ}。Lec 10 回答它们何时合法，答案是"**策略变化不太大时**"，由此引出 TRPO。

11.2 策略梯度 = 策略迭代

$$\pi_{\text{new}} = \arg\max_\pi \mathbb{E}_{s\sim d^{\pi_{\text{old}}}}\left[\mathbb{E}_{a\sim\pi}[A^{\pi_{\text{old}}}(s,a)]\right].$$

不限距离时最优解是"把所有概率给 $\arg\max_a A$"——即贪心策略迭代；一旦走这么远，d^π 与 $A^{\pi_{\text{old}}}$ 的近似立即失效。

分布偏移的界（【讲义】Lec 10，证明基于 TRPO 论文）：

$$\|\mathbb{E}_{s\sim d^{\pi_{\text{new}}}}[f] - \mathbb{E}_{s\sim d^{\pi_{\text{old}}}}[f]\| \le 2\epsilon\,\mathbb{E}_{s\sim d^{\pi_{\text{old}}}}\left[D_{\text{TV}}(\pi_{\text{new}}\|\pi_{\text{old}})(s)\right],$$

其中 $\epsilon = \max_s \left|\mathbb{E}_{a\sim\pi_{\text{new}}}[A^{\pi_{\text{old}}}(s,a)]\right|$。直觉：单状态上的策略改变会被**访问频率**放大；每步偏移 $O(\delta)$，T 步可累积到 $O(T\delta)$（与 BC 的 $O(\epsilon T^2)$ 同源）。

11.3 单调改进下界

$$J(\pi_{\text{new}}) \ge J(\pi_{\text{old}}) + \mathbb{E}_{s\sim d^{\pi_{\text{old}}}}\left[\mathbb{E}_{a\sim\pi_{\text{new}}} A^{\pi_{\text{old}}}\right] - \frac{2\epsilon\gamma}{(1-\gamma)^2}\mathbb{E}_s\left[D_{\text{KL}}(\pi_{\text{new}}\|\pi_{\text{old}})(s)\right].$$

最大化下界即保证 J 不下降。把惩罚换成**硬约束**（因为 $\frac{2\epsilon\gamma}{(1-\gamma)^2}$ 太大、难调）：

$$\max_\theta \mathbb{E}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\text{old}}(a\mid s)}A^{\pi_{\text{old}}}(s,a)\right] \quad \text{s.t.} \quad \mathbb{E}_s\left[D_{\text{KL}}(\pi_\theta\|\pi_{\text{old}})(s)\right]\le\delta.$$

11.4 自然梯度

为什么普通梯度不够：参数空间的欧氏距离与策略分布距离不是一回事——同样的 $\|\Delta\theta\|$，在"敏感方向"造成巨大 KL，在"不敏感方向"几乎没变。最速方向应按 KL 曲率定义。

一阶 Taylor：$L\approx L_0+g^\top\Delta\theta$；KL 在 θ_0 取极小，故一阶项为 0，二阶为 $\frac{1}{2}\Delta\theta^\top F\Delta\theta$，
$F=\mathbb{E}[\nabla\log\pi_\theta\nabla\log\pi_\theta^\top]$ 是 **Fisher 信息矩阵**。在 KL 椭球内最大化线性目标，得

$$\boxed{\theta\leftarrow\theta+\alpha F^{-1}g}$$

TRPO 实际算法（【讲义】Lec 10）：算 g → 用**共轭梯度（CG）** 解 $Fx=g$（只需 $O(n)$ 次 Hessian-vector product，不显式构造 F）→ 线搜索步长使 KL 满足且目标改进 → 更新。

```
g = policy_gradient(surrogate)
x = conjugate_gradient(Avp_f, g, nsteps=10)
shs = 0.5 * (x @ Avp_f(x)); lm = sqrt(0.5 * max_kl / shs)
step = lm * x
for i in range(10):                          # 线搜索
    new = params + (0.5**i) * step
    if surrogate_improved(new) and kl(new) <= max_kl: break
```

11.5 TRPO vs PPO

维度	TRPO	PPO-Clip
约束	硬 KL 约束（CG + 线搜索）	软裁剪（一阶）
每步代价	高（Hessian-vector product）	低（普通 SGD）
步数	一大步	很多小步
稳定性	理论保证强	实践够用
现代使用	少（研究基线）	**主流**

讲义总结：**"TRPO 是一大步，PPO 是很多小步"**；PPO 赢在"工程复杂度 vs 稳定性"的折中。

11.6 实现与失败模式

TRPO 用 $\delta=0.01$，诊断量是 KL/surrogate/线搜索接受率，常见 bug 是 CG 的 damping 太小导致数值不稳。PPO 实践中若 $\mathrm{KL}>2\delta$ 就提前停止该 epoch 更新（early stopping），是很有效的稳定手段。

现象	根因
KL 超限仍接受步 → 崩	线搜索判据写错
数值不稳	CG damping 太小、$\boldsymbol{F}$ 近似误差
PPO 后期震荡	clip 太松、β 太小

第 12 章 变分推断与控制即推断（Lec 11–13）

12.1 为什么 RL 课要讲变分推断

Lec 11–13 是理解 **SAC、IRL、扩散策略、世界模型**的枢纽，回答三问：多模态策略怎么建模？"加熵"为什么有原理？逆 RL 的似然怎么算？

12.2 潜变量模型与 ELBO

$$p_\theta(x) = \int p_\theta(x \mid z) p(z) dz.$$

两种用法（**【讲义】Lec 11**）：**多模态策略**（z 是模式/风格，$\pi_\theta(a \mid s, z)$）；**世界模型**（z 是隐状态）。

引入近似后验 $q_\phi(z \mid x)$，由 Jensen 不等式：

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] - D_{\mathrm{KL}}\left(q_\phi(z \mid x) \| p(z)\right).$$

讲义强调这是"不正确但非常方便"的下界。**摊销（amortized）推断**用一个网络直接输出后验参数；**重参数化** $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$ 让梯度穿过采样。两种梯度对比：score-function 可处理离散/连续但方差大；重参数化只适用连续但方差小、实现简单。

与扩散的关系（**【讲义】Lec 12**）：扩散 = **层级化 VAE**（前向加噪是固定编码器，反向去噪是学习到的解码器）。这解释了扩散策略为何能表达多模态。

12.3 控制即推断

引入最优性变量 $\mathcal{O}_t$，$p(\mathcal{O}_t = 1 \mid s_t, a_t) = \exp(r(s_t, a_t))$：

$$p(\tau, \mathcal{O}_{1:T}) = p(s_0) \prod_t p(s_{t+1} \mid s_t, a_t) \pi(a_t \mid s_t) \exp(r(s_t, a_t)).$$

没有假设行为最优——$\mathcal{O}$ 只是"证据"变量。定义反向消息 $\beta_t(s_t, a_t) = p(\mathcal{O}_{t:T} \mid s_t, a_t)$，则

$$\beta_t(s_t, a_t) = \exp(r(s_t, a_t)) \, \mathbb{E}_{s'}\left[\beta_{t+1}(s')\right], \qquad \beta_{t+1}(s') = \mathbb{E}_{a \sim \pi}[\beta_{t+1}(s', a)].$$

取对数（$V = \log \beta$）得**软值迭代**：

$$\boxed{V_t(s) = \log \int \exp(Q_t(s,a))da} \qquad (\text{离散时是 log-sum-exp})$$

对比硬 Bellman 的 $\max_a$：**soft max 替代 max**，这就是 soft optimality。策略即后验：

$$\pi(a_t \mid s_t, \mathcal{O}_{t:T}) = \exp\left(Q_t(s_t,a_t) - V_t(s_t)\right) = \exp\left(A_t(s_t,a_t)\right),$$

即**优势的指数**就是最优动作的后验——这正是 **Boltzmann 策略**，也解释了"为何随机策略可能最优"。

乐观问题（optimism problem）（**【讲义】Lec 13**）：朴素反向消息会偏好"能到达高价值状态"的动作，**即使那个转移靠运气**（极端情况会去撞随机传送门）。修正为 variational control：

$$\log p(\mathcal{O}_{1:T}) \geq \mathbb{E}_{\tau\sim q}\left[\sum_t r(s_t,a_t) + \mathcal{H}(\pi) - \log p(s_{t+1} \mid s_t, a_t)\right],$$

即在奖励外**加熵**、并减去环境随机性。**熵不是正则技巧，而是变分下界的产物**（详见 Levine 2018 tutorial）。

12.4 从 soft optimality 到 SAC

组件	公式
软 Q 备份	$Q = r + \gamma \mathbb{E}_{s'}[V(s')]$，$V(s') = \mathbb{E}_{a'\sim\pi}[Q(s',a') - \alpha \log \pi]$
策略改进	$\pi \leftarrow \arg\max_\pi \mathbb{E}[Q - \alpha \log \pi]$，解为 $\pi \propto \exp(Q/\alpha)$
实践	高斯策略 + 重参数化近似"能量策略"

注意：理论最优 $\exp(Q/\alpha)$ 可能是多峰的，而 SAC 用单峰高斯——这是其表达力瓶颈，也是后来"扩散策略 + Q 学习"（IDQL、FQL）要解决的问题（见 14.6）。

12.5 实现与失败模式

项目	建议	说明
KL 权重	从小（1e−3）开始 + warm-up	太大会 posterior collapse
后验方差	`logvar` clamp 到 $[-10, 2]$	防数值爆炸
多模态	z 维数 ⩾ 模态数	太少表达不了
诊断	看 $q(z \mid x)$ 是否退化为先验	posterior collapse

第 13 章 Model-Based RL：学习一个模拟器

13.1 动机

样本效率的终极解法：**学模型 $p_\theta(s' \mid s, a)$，在"想象"里训练**。Lec 15 给三类动机（**【讲义】Lec 15**）：样本效率（真机贵）、安全性（在模型里试错不炸机器人）、可解释/可规划。

13.2 朴素算法与致命问题

```
D = collect_random_data()
repeat:
    model  = fit_model(D)                 # 监督学习 (s,a) -> s'
    policy = rl_in_model(model)           # 在模型里跑任意 model-free RL
    D += rollout(policy, real_env, n)     # 少量真实数据
```

分布偏移（又是它）：模型只在训练分布上准；策略优化会**主动寻找模型的高误差区域**（那里"看起来"奖励高）。模型误差 ϵ_m、rollout 长度 H 时，回报误差可达 $O(\epsilon_m H^2)$，与 BC 的 $O(\epsilon T^2)$ 同构，但更糟：**误差被策略主动放大**。

三条缓解路径：① 限制策略变化（trust region）；② 只信模型"有信心"处（不确定性惩罚/悲观）；③ 短 rollout + 自举（Dyna、MBPO）。

13.3 不确定性估计

【讲义】Lec 15 明确区分：

类型	别名	来源	输出熵能否估计
偶然不确定性	aleatoric / statistical	环境本身随机性	能（输出方差）
认知不确定性	epistemic / model	"我们不知道模型参数"	**不能**

关键洞察：**输出熵只反映 aleatoric**。若模型在某个状态"自信地给出错误预测"，输出熵很低却错得离谱。要估 epistemic，需要**模型集合**或 BNN。

Bootstrap ensemble（讲义推荐的最实用方案）：

```
for i in range(N):                        # N = 5~10
    # 不同初始化 + 不同数据子集；重采样常可省略（SGD+随机初始化已足够独立）
    model_i = fit(D_sample_i)
preds = [model_i(s,a) for i in range(N)]
mean, var = preds.mean(), preds.var()    # var 即 epistemic 估计
```

评审要点：$N < 10$ 时方差估计粗糙，但经验上足够用于"惩罚"——因为只需**排序**而非精确值。其他方案：BNN + MC dropout（更贵）、probabilistic ensemble（PETS，每模型输出高斯）。

13.4 规划：MPC 与 CEM

术语（【讲义】Lec 16）：**开环** = 一次算好整段动作、不看反馈执行；**闭环** = 每步重规划（receding horizon）。开环在随机环境次优；**MPC = 只执行计划第一步再重规划**，用重规划把开环规划器变成闭环控制器。

```
mu, sigma = zeros(H, A), ones(H, A)
for iter in range(n_iter):
    A_samples = mu + sigma * randn(N, H, A)          # 1. 采样动作序列
    returns = [model_return(s0, a) for a in A_samples]   # 2. 模型 rollout
    elite = A_samples[argsort(returns)[-k:]]         # 3. 精英重估分布
    mu, sigma = elite.mean(0), elite.std(0) + eps
    # 执行 mu[0]，然后重规划（receding horizon）
```

优点	缺点
极简、可并行、不需梯度	维度上限约 40 维（讲义原话）；只能开环（除非配 MPC）

替代：CMA-ES（带动量的 CEM）、MCTS（离散）、LQR/iLQR（可微连续）。**不确定性下的规划**：从 ensemble 抽模型 rollout 多次取平均（moment matching），或用 $\mu - \beta\sigma$ 做悲观规划。

13.5 Dyna 与 MBPO：短 rollout + 自举

Dyna（【讲义】Lec 16，Sutton 1990）：

```
a = eps_greedy(Q, s); s2, r, d = env.step(a)
Q.update(s, a, r, s2); model.update(s, a, r, s2)
for _ in range(n_planning):                  # 用模型"想象"
    s_p, a_p = sample_from_buffer()
    s2_p, r_p = model.predict(s_p, a_p)
    Q.update(s_p, a_p, r_p, s2_p)
```

关键：**模型只滚 1 步**——误差不累积，且起点来自真实 buffer 的多样状态。

MBPO（Janner et al. 2019）的**分支 rollout**：

```
for each real env step:
    D_real.add(transition); model = fit_ensemble(D_real)
    for s0 in sample(D_real, k):             # 从真实数据采起点
        for h in range(H):                   # H 很小（1~5）
            a ~ pi(s); s', r = model(s, a); D_model.add(...)
    sac_update(D_real ∪ D_model)
```

为什么好	为什么坏
H 小 → 误差不累积	看不到 H 步之后的真实状态（长程信用分配靠 Q 自举）
起点来自真实分布 → 覆盖多样状态	模型误差仍通过 Q 传播

超参：H 通常**随训练逐渐增大**（早期 $H = 1$，后期增大）。

13.6 潜空间世界模型

高维观测下直接在像素上建模太贵。学压缩表示 z_t，在 z 上做动力学：

$$q_\phi(z_t \mid o_{\le t}) \text{（编码器）}, \quad p_\theta(z_{t+1} \mid z_t, a_t) \text{（动力学）}, \quad p_\theta(o_t \mid z_t),\ p_\theta(r_t \mid z_t, a_t).$$

编码器	精度	复杂度	说明
全平滑后验	最高	最复杂	用整段轨迹推断 z_t
单步编码器	最低	最简单	$q(z_t \mid o_t)$，忽略历史
滤波后验	中	中	$q(z_t \mid o_{\le t})$，RNN/Transformer

坑：单步编码器在部分可观测时丢信息；确定性编码器无法表达"多峰信念"。训练目标 = 重建 + 奖励 + 动力学一致性 KL：

$$\mathcal{L} = \mathbb{E}[\log p(o_t \mid z_t)] + \mathbb{E}[\log p(r_t \mid z_t, a_t)] - D_{\mathrm{KL}}\left(q(z_t \mid o_{\le t}) \| p(z_t \mid z_{t-1}, a_{t-1})\right).$$

代表工作【补充】：World Models（Ha & Schmidhuber 2018）、Dreamer/DreamerV3（潜空间 actor-critic + 离散潜变量）、PlaNet/PETS（概率 ensemble + MPC）、TD-MPC2、MuZero（只学"预测奖励/价值/策略"的抽象模型，不重建观测）。

13.7 偏差–方差：model-based 的独特之处

与 model-free 相比，model-based 的偏差–方差结构**方向相反**，这是最容易搞混的一点：

量	model-free	model-based
方差	高（真实环境的随机性 + MC 回报）	**低**（模型是确定/平滑的，等于"去噪"）
偏差	低到中（无偏 MC 或自举偏差）	**高**（模型误差 ϵ_m 是系统性的）
误差随 horizon	$O(\sqrt{H})$（MC 方差）	$O(\epsilon_m H^2)$（**偏差累积**）

三条推论：

1. **模型越准，不代表策略越好**，因为策略会主动把状态推到"模型误差最大"的区域；真正决定成败的是**策略访问分布上的模型误差**。
2. **rollout 越长，偏差累积越快**（H^2），所以 MBPO 用 $H = 1 \sim 5$ 并从真实状态分支。
3. **ensemble 不是降偏差，而是把"偏差"变成"可检测的方差"**：$\mathrm{Var}[\text{ensemble}]$ 是 epistemic 的可估代理，用它做惩罚（pessimism）等价于"在不确定处假设最坏情况"，把偏差转成保守性。

13.8 失败模式

现象	根因	处理
策略"利用"模型漏洞	模型误差被优化器放大	ensemble 惩罚、短 rollout、trust region

现象	根因	处理
模型 loss 降但策略变差	关键区域仍错	按策略访问分布加权采样
像素任务学不动	重建目标与控制特征不匹配	MuZero 式无重建目标、auxiliary loss
MPC 抖动	每次重规划解不连续	动作平滑惩罚、warm-start
规划维度上不去	CEM 高维失效	梯度（iLQR）或降维

第 14 章 离线 RL：不用交互也能改进策略

14.1 动机

真实场景常只有一份固定数据（医疗、驾驶日志、推荐），不能再交互。离线 RL（offline / batch RL）要回答：**给定 $\mathcal{D} = \{(s, a, r, s')\}$，能否学出比数据里最好行为更好的策略？** 【讲义】Lec 17 给三条理由：找好数据、泛化、拼接（stitching，macro/micro-scale）。

14.2 核心困难：反事实查询

$$Q(s,a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

的 $\max$ 会查询**数据里没出现过的** (s', a')。在线时你可以去试，离线时只能靠函数逼近**外推**——而神经网络的外推是任意的（通常高估）。讲义用 SAC 实验说明：数据越少，Q 值高估越离谱（"how well it thinks it does" 与 "how well it does" 在对数坐标差几个数量级）。

误差	在线 RL	离线 RL
采样误差	有	有，且**更严重**
函数逼近误差	有	有，且**被 $\max$ 放大**
分布偏移	有（可交互修正）	**无法修正**

14.3 三条技术路线

路线	做法	代表
策略约束	让策略贴近数据行为	SAC+BC、TD3+BC、BRAC、AWAC、BCQ
保守性	让 Q 对 OOD 动作**低估**	CQL
避免 OOD 查询	备份时只用数据里的动作	IQL

BCQ（Batch-Constrained Q-learning）【补充】（Fujimoto et al. 2019）是"策略约束"的另一条实现路径：先用一个**生成模型**（VAE）拟合行为策略 π_β，再学一个**扰动网络** $\xi_\phi(s,a)$，把动作限制在"生成模型的重建 + 小幅扰动"内：

$$a' = \mu_\beta(s') + \xi_\phi(s', a), \qquad |\xi| \le \Phi \text{（小常数）},$$

然后在这个**受限集合**里取 max：$y = r + \gamma \max_{a' \in \text{BCQ-set}} Q_{\phi'}(s', a')$。与 CQL 的区别是"**从候选集上做约束**"而非"改损失"：它从不查询生成模型支持之外的动作。缺点是多了一套生成模型 + 扰动网络，超参（Φ、VAE latent 维度）更敏感；优点是"约束"的语义非常直观。

14.4 策略约束：SAC+BC 与 AWAC

SAC+BC（HW5 PDF 式 (3)）：

$$L(\pi) = \mathbb{E}_{(s,a)\sim\mathcal{D},\, a^\pi\sim\pi}\left[-\frac{1}{2}\sum_{i=1}^{2} Q_i(s, a^\pi) + \alpha \cdot \frac{1}{|\mathcal{A}|}\|a^\pi - a\|_2^2 + \beta \log \pi(a^\pi \mid s)\right].$$

三项分别是"最大化 Q / BC 正则 / 熵"，α 是最重要超参：太小 OOD 崩，太大退化为 BC。**判断 α 的技巧**（HW5 Tips，极实用）：监控"数据集动作与策略动作的 MSE"，好的 α 使 MSE **略高于**纯 BC 的最小 MSE；若 MSE 随 α 增大明显下降，说明 BC 项在起作用。另可用 `q_min/q_max` 做 sanity check：本作业奖励为 $\{-1, 0\}$，故 Q 应落在 $[-\frac{1}{1-\gamma}, 0]$（antmaze/antsoccer 上 q_max→0、q_min→−100；cube-single 因短程可能只到 −50～−70）。

前向 KL vs 反向 KL（**【讲义】Lec 17**）：

约束	公式	行为	适用
前向 KL	$D_{\mathrm{KL}}(\pi_{\text{data}} \| \pi_\theta)$	mode-covering（覆盖所有模态）	想"不遗漏"
反向 KL	$D_{\mathrm{KL}}(\pi_\theta \| \pi_{\text{data}})$	mode-seeking（只学一个模态）	**离线 RL 更常用**

离线 RL 偏好反向 KL：它**不强迫保留低奖励模态**，只惩罚"数据里根本不可能的动作"。理想的是 support constraint（$\pi(a \mid s) > 0 \Rightarrow \pi_{\text{data}}(a \mid s) > 0$），但难精确实现，反向 KL 是可算近似。

AWAC（**【讲义】Lec 18**）：带 KL 约束的策略优化，其解是**优势加权回归** $\pi^\star \propto \pi_{\text{data}} \exp(A/\lambda)$，故可直接做加权最大似然：

$$L(\pi) = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\exp(A(s,a)/\lambda)\log \pi(a \mid s)\right].$$

+ 无需对抗训练、无需估 behavior policy；**−** 权重指数易爆（要 clip），对 A 的误差敏感。

14.5 IQL：完全不查 OOD 动作

【讲义】Lec 18（Kostrikov et al. 2021）：既然 $\max$ 是万恶之源，能否**不查任何 OOD 动作**也算出"数据支持的最优值"？答案是用**expectile 回归**近似 max：

$$L(V) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\ell_2^\tau\left(V(s) - \min_i Q_i(s,a)\right)\right], \qquad \ell_2^\tau(x) = |\tau - \mathbb{1}(x > 0)|x^2.$$

- $\tau = 0.5$ → 对称平方损失 → 学到**均值**（≈SARSA/行为策略的值）。

- $\tau \to 1$ → 只惩罚低估 → 学到**上分位数** $\approx \max_{a \sim \mathcal{D}}$。
- 关键：**V 只看 s，Q 的自变量 a 全部来自数据**，无任何 OOD 查询。

完整三个损失（HW5 式 (5)–(8)）：

$$
\begin{aligned}
L(Q) &= \sum_{i=1}^{2} \mathbb{E}\left[(Q_i(s,a) - r - \gamma V(s'))^2\right], \\
L(V) &= \mathbb{E}\left[\ell_2^\tau\left(V(s) - \min_i \bar{Q}_i(s,a)\right)\right], \\
L(\pi) &= \mathbb{E}\left[-\min\left(\exp(\alpha A(s,a)), M\right) \log \pi(a \mid s)\right], \quad A = \min_i Q_i - V.
\end{aligned}
$$

HW5 固定 $\tau = 0.9$、$M = 100$，只调 α（建议从 $\{1, 3, 10\}$ 起）。超参语义：$\tau = 0.5$ 是 SARSA 算子、$\tau \to 1$ 是 Bellman 最优算子；$\alpha = 0$ 是纯 BC、$\alpha \to \infty$ 是"数据动作上的贪心"。**为什么稳**：policy improvement 藏在 V 的 expectile 里，策略只做加权 BC，永不查询 OOD。

14.6 CQL 与表达力：从高斯到 flow

CQL（**【讲义】Lec 18**，Kumar et al. 2020）：不约束策略，而是**修正 Q**——对 OOD 动作压低、对数据动作抬高：

$$
L_{\text{CQL}} = \alpha\, \mathbb{E}_{s \sim \mathcal{D}}\left[\log \sum_a \exp Q(s,a) - \mathbb{E}_{a \sim \mathcal{D}} Q(s,a)\right] + \frac{1}{2}\mathbb{E}_{(s,a,r,s')}\left[(Q - \mathcal{B}Q)^2\right].
$$

第一项是对"所有动作"的 soft-max（压 OOD），第二项把数据动作的 Q 抬上来。理论上给出 Q^π 的下界（α 足够大时），故"低估→保守→安全"。实现要点：连续动作用重要性采样估 $\log \sum \exp$（uniform + 当前策略 + 数据动作）；α 常用 Lagrange 自动调；**过度保守**会让策略"什么都不做"（offline-to-online 时尤甚）。

表达力问题（**【讲义】Lec 18 + HW5 PDF**）：离线行为策略常是多峰的，单峰高斯既表达不了、也"约束不好"。

方法	做法	代价
FBRAC	SAC+BC 的 BC 项换成 flow matching	需对 ODE 做 BPTT（贵、不稳）
FQL	另学一个**一步策略** $\pi_\omega(s,z)$，Q 最大化只用它；flow 只做 BC	无 BPTT
IDQL	IQL 学 Q + 扩散做 BC，推理采样 N 个候选按 Q 排序	推理贵
Diffusion steering	在扩散模型潜空间跑 SAC	需好编码器

FQL 三损失（HW5 式 (13)–(17)）：

$$
\begin{aligned}
L(v) &= \mathbb{E}\left[\tfrac{1}{|\mathcal{A}|}\left\|v(s,\tilde{a},t) - (a - z)\right\|^2\right], \quad \tilde{a} = (1-t)z + ta, \\
L(Q) &= \sum_i \mathbb{E}\left[(Q_i(s,a) - y)^2\right], \quad y = r + \tfrac{\gamma}{2}\sum_j \bar{Q}_j\left(s', \pi_\omega(s',z)\right), \\
L(\pi_\omega) &= \mathbb{E}\left[-\tfrac{1}{2}\sum_i Q_i(s,\pi_\omega(s,z)) + \tfrac{\alpha}{|\mathcal{A}|}\left\|\pi_\omega(s,z) - \pi_v(s,z)\right\|^2\right].
\end{aligned}
$$

关键设计：Bellman 备份用**一步策略**（前馈网络，无 BPTT），蒸馏目标 π_v（多步 ODE）**不回传梯度**。HW5 两个易错点：喂 Q 的 flow 动作要 **clip 到 $[-1, 1]$**；但**蒸馏项里的一步策略动作不要 clip**，否则无法被纠正回合法范围。

决策 Transformer（Decision Transformer）【补充】（Chen et al. 2021）走的是完全相反的路线：**把离线 RL 当成序列建模**，不学值函数、不做自举。把轨迹重写成

$$\tau = \left(\hat{R}_1, s_1, a_1,\ \hat{R}_2, s_2, a_2,\ \ldots\right), \qquad \hat{R}_t = \sum_{t' \geq t} r_{t'},$$

用因果 Transformer 在"**已达成回报（return-to-go）**"的条件下预测 a_t：

$$L(\theta) = \frac{1}{T} \sum_t \left\| \pi_\theta\left(\hat{R}_{1:t}, s_{1:t}, a_{<t}\right) - a_t \right\|^2.$$

优点	缺点
无自举，训练极稳（纯监督）	**拼接能力弱**（不能把"好片段"组合成新策略，因为它模仿的是轨迹统计）
可用"把 $\hat{R}$ 调高"来引导更好的行为	依赖回报的条件分布，OOD 回报可能失效
实现简单，天然支持长序列/多模态	不能像 Q 方法那样做"策略改进"

它的理论意义大于工程意义：说明"**RL 的稳定性问题可能来自自举与 max**"，若改成纯监督序列建模，稳定性立刻变好，代价是失去"改进"能力。这与第 14.5 节 IQL"避免 OOD 查询"、14.6 节 CQL"保守性"是同一问题的三种解法。

14.7 离线到在线

【讲义】Lec 18 讲了一个"令人不适"的现象：CQL 类方法离线学得好，切到在线却要**先浪费一段时间**把过度保守的 Q 修正回来。更"令人不适"的结论（讲义称 "an embarrassingly effective method"，Ball et al. 2023）：

> 把离线数据与在线 replay **各取一半混合训练**、且**不做任何预训练**，就能打败大多数"正统"的 offline-to-online 方法。

含义：**离线 RL 的最大价值可能是"提供覆盖广的数据"，而非"预训练一个策略"**。另外讲义提到，用扩散/flow 做 actor 的方法在 offline-to-online 上意外地好（作者坦言"不知道为什么"），可能因为"不必精确捕捉单个模态"。

14.8 失败模式

现象	根因	处理
Q 值飞到 10^3	OOD 高估	加 CQL/BC、降 lr、降 UTD
策略"躺平"	过度保守	减小 α、增加数据覆盖
学成 BC	约束太强	减小 BC 系数
多模态只学一个峰	高斯策略	换 flow/扩散策略
离线好、在线崩	值函数过度保守	混合采样、重置 optimizer
seed 间方差巨大	数据覆盖不足	多 seed、检查数据多样性

第 15 章 探索：从 ϵ–greedy 到 RND

15.1 动机：为什么探索"理论上不可解、实践上必须做"

【讲义】Lec 19 用《蒙特祖玛的复仇》说明问题：拿钥匙/开门是奖励，被骷髅碰死**没有奖励**（是好是坏？），通关与局部奖励只有**弱相关**。人类知道该做什么，是因为理解 sprite 的语义。讲义的类比是玩一局"规则不告诉你、只能试错、且规则本身不讲道理"的桌游（Mao）："如果人生的目标是赢 50 局 Mao，而且你事先不知道这件事"——这就是 RL 的探索问题。

两种等价表述（讲义明确指出是**同一问题的两面**）：① **时间延展**——如何发现"需要一长串本身无奖励的行为"才拿到高奖励的策略？② **探索–利用**——尝试新行为还是继续做已知最好的？

可解性谱系（**【讲义】Lec 19**）：多臂老虎机、上下文老虎机、小表格 MDP **可解**；大 MDP + 已知低秩特征**部分可解**；大 MDP 无特殊结构**理论不可解**（需指数级样本）。关键结论：**最优探索策略 = 解一个 POMDP（把"未知"当隐状态）**，但信念状态太大不可算，故实践用"启发式 + 从小规模最优方法借灵感"。

15.2 经典方法

ϵ–greedy：以 $1-\epsilon$ 贪心、ϵ 均匀随机。零成本、任何算法可加；但**探索盲目**（高维动作几乎撞不到有用动作）、**不随状态变化**（已探索处还在浪费）。实践：ϵ 从 1.0 退火到 0.05–0.1，**不要退到 0**。

Boltzmann 探索：$\pi(a \mid s) = \exp(Q(s,a)/\tau) / \sum_{a'} \exp(Q(s,a')/\tau)$。按价值排序分配探索，但 τ 难调、Q 尺度变化会改变有效温度。注意：**这正是第 12 章"控制即推断"给出的最优策略形式**（$\tau = 1$ 时 $\pi \propto \exp Q$）。

乐观探索（UCB）：

$$a_t = \arg\max_a \left[\hat{Q}(a) + \beta \sqrt{\frac{\log t}{N(a)}} \right].$$

直觉是"没试过的动作上界很高"，故先去试直到确定它不好。MDP 里把 bonus 加到奖励：$r^+ = \beta / \sqrt{N(s,a)}$（MBIE-EB）或 $\beta\sqrt{\log t / N}$（UCB）。**问题**：高维/连续状态下 $N(s,a)$ 永远是 0 或 1——"计数诅咒"。

15.3 伪计数与 RND

伪计数（pseudo-count）（**【讲义】Lec 19**，Bellemare et al. 2016）用密度模型 ρ_θ 定义

$$\hat{N}(s) = \frac{\rho_\theta(s)\,(1-\rho'_\theta(s))}{\rho'_\theta(s) - \rho_\theta(s)}, \qquad r^+(s) = r(s) + \beta / \sqrt{\hat{N}(s)},$$

其中 ρ'_θ 是"没见过当前样本"的密度（需要重训一次）。要求是**只要能输出密度**，不需要采样质量（与 GAN 取向相反）。代价是每 batch 训两次；效果是在 Montezuma 上从"几乎 0 分"提到数千分。

用预测误差当新奇度：$r^+(s) = \|\hat{f}(s) - f(s)\|^2$。**致命问题**是误差也来自环境随机性（aleatoric），智能体会去追"噪声电视"（noisy TV problem）。

RND（Random Network Distillation）（Burda et al. 2018）的解法：目标网络**随机初始化且冻结**，

$$r^+(s) = \left\| \hat{f}_\theta(s) - f(s) \right\|^2.$$

因 f 确定性，同一状态永远给同一目标 → 随机噪声不产生误差；$\hat{f}_\theta$ 被训练拟合 f，"见过的"误差小、"没见过的"误差大，且误差随访问自然衰减。

```
f = RandomNet(); freeze(f); f_hat = RandomNet()
while training:
    r_int = ||f_hat(s) - f(s)||**2
    r_total = r_ext_norm + beta * r_int_norm    # 两路分别归一化!
    loss = mse(f_hat(s), f(s).detach())         # 只训 f_hat
```

RND 的三个关键 trick：**外在/内在奖励分别做 running std 归一化**；**两个价值头**（V_{ext}、V_{int}，因折扣与量级不同）；β 需退火（否则后期还在追新奇）。

15.4 技能发现与互信息（Lec 23）

【讲义】Lec 23 的问题：完全不知道奖励时能学什么？答案是一组多样化技能，用途有三：学技能不需监督、可作分层 RL 的子策略、可用于探索行为空间。

DIAYN（Eysenbach et al. 2018）：

```
z ~ p(z)                              # 技能 latent
每步: a ~ pi(a | s, z)
      r = log q(z | s) - log p(z)     # 互信息 I(S;Z) 的变分下界
```

目标 $\max I(S;Z)$：不同 z 访问不同状态区域。实现用判别器 $q(z \mid s)$ 估下界。结果：不同 z 学到不同步态（Cheetah 跑/跳/翻滚，Ant 各种走法）。

目标条件 + Skew-Fit：把 z 换成目标 g、判别器变"预测目标"，即目标条件技能；Skew-Fit（Nair/Dalal et al. 2019）按"当前成功率反比"重加权采样目标，自动扩展目标分布。

其他范式【补充】：Go-Explore（存档好状态再探索，解决 detachment）、HER（失败轨迹重标注）、ICM（前向模型误差当 curiosity）、Never Give Up（多 episodic 探索）。

15.5 失败模式

现象	根因	处理
奖励被"噪声"吸走	noisy TV	用 RND 而非前向预测误差
后期不收敛	bonus 没退火	退火 β
内在/外在量级失衡	没分别归一化	两个 running std
学了技能下游没用	技能不可组合/不可辨识	加目标条件、解耦约束
ϵ-greedy 在高维动作下无效	均匀随机太浪费	参数噪声（NoisyNet）、熵正则

第 16 章 RL 理论：误差如何复合

16.1 先摆正预期

【讲义】 Lec 20 开篇的"免责声明"值得引用：**不要相信任何声称自己的 RL 算法有"可证明保证"的人——假设总是不现实的，理论最多是"可能发生什么"的粗略指引**。理论有三个用途：证明理想条件下收敛；理解误差如何随问题参数缩放（horizon、γ、样本量）；指导"该往哪改算法"。

16.2 值迭代收敛

Bellman 最优算子 $(\mathcal{T}V)(s) = \max_a [r(s,a) + \gamma \mathbb{E}_{s' \sim p} V(s')]$ 是 γ–压缩：

$$\|\mathcal{T}V_1 - \mathcal{T}V_2\|_\infty \le \gamma \|V_1 - V_2\|_\infty$$

（证明用 $|\max_a f - \max_a g| \le \max_a |f - g|$ 再提 γ）。由 Banach 不动点定理，

$$\|V_k - V^\star\|_\infty \le \frac{\gamma^k}{1-\gamma} \|V_0 - V^\star\|_\infty.$$

注意：证明只用"精确备份"。一旦引入函数逼近（投影），压缩性被破坏（见 7.4）。

16.3 样本复杂度（无探索的简化设定）

【讲义】 Lec 20 的设定：可任意查询 (s,a) 的下一个状态（generative model，可重置），于是不需要"探索"，只需估出 $Q^\star$。工具是 Hoeffding + union bound + 三角不等式。单点用 n 个样本估 $\hat{Q}$ 时，以概率 $1-\delta$：

$$\|\hat{Q} - Q^\star\|_\infty \le \epsilon_{\text{sample}} = O\left(\sqrt{\frac{\log(|\mathcal{S}||\mathcal{A}|/\delta)}{n}}\right).$$

K 步迭代后：

$$\|\hat{V}_K - V^\star\|_\infty \le \underbrace{\frac{\gamma^K}{1-\gamma}\|V_0 - V^\star\|_\infty}_{\text{初始误差衰减}} + \underbrace{\frac{\epsilon_{\text{sample}}}{1-\gamma}}_{\text{误差累积}}.$$

每次备份把误差乘 γ 并累加，几何级数求和给出 $\frac{1}{1-\gamma}$ 放大——**$\gamma \to 1$ 时误差线性爆炸**。若要求总误差 $\le \epsilon$，需 $n = O\left(\frac{\log(|\mathcal{S}||\mathcal{A}|/\delta)}{(1-\gamma)^4 \epsilon^2}\right)$，即 **$(1-\gamma)^{-4}$ 是"长程任务的诅咒"**。

16.4 FQI 的误差分解

$$\|\hat{Q} - Q^\star\|_\infty \lesssim \underbrace{\frac{\epsilon_{\text{approx}}}{1-\gamma}}_{\text{函数类表达力}} + \underbrace{\frac{\epsilon_{\text{sample}}}{(1-\gamma)^2}}_{\text{采样}}.$$

采样项是二阶极点，因为每次备份同时引入采样误差、而误差又被 γ 累积。降低手段分别对应"更大网络/更好特征"与"更多数据/更好覆盖"。

16.5 理论到工程的翻译

理论结论	工程含义
γ–压缩只在精确备份下成立	target network / replay 是在"假装"精确备份
误差 $\sim (1-\gamma)^{-2}$	长程任务必须更大网络、更多数据、更稳算法
最坏情况探索不可解	别指望"通用探索算法"，要用领域先验
表格型样本复杂度是多项式	深度 RL **没有**这样的保证
悲观/乐观需要假设	CQL 的下界依赖"数据覆盖"假设

研究陷阱：把"渐近收敛"当"有限样本可用"（常数可能大到无意义）；忽略探索假设；∞–范数最保守而 p–范数更现实但更难证；低估分布偏移。

第 17 章 RLHF 与推理 RL：把 RL 用在语言模型上

17.1 动机：LLM 为什么需要 RL

【讲义】Lec 14 把训练分三段：

```
预训练（next-token prediction，海量多样无标注文本）
   → 后训练（SFT / instruction tuning，高质量窄数据）
   → RL 微调（RLHF 人类偏好 / RLVR 可验证奖励）
```

核心论断：**预训练给世界知识，后训练告诉模型怎么用**；预训练完的模型不是 agent，只是"文本补全引擎"。后训练两条路线：**RLHF**（奖励来自人类偏好→奖励模型；InstructGPT、Claude）与 **RLVR**（奖励可自动验证：数学、代码；DeepSeek–R1、o 系列）。

17.2 LLM 生成作为 MDP

MDP	LLM
状态 s_t	prompt + 已生成 token $(x, y_{<t})$
动作 a_t	下一个 token（词表 V）
转移	**确定性**（拼接）
奖励	通常只在最后 token 给（稀疏），也可给过程奖励

MDP	LLM
折扣	$\gamma = 1$（或按长度）

两种等价视角：**单步 bandit**（整段回答是一个动作，奖励 $r(x, y)$，适合 REINFORCE/GRPO）与**多步 MDP**（每 token 一个动作，适合 PPO+GAE+值函数）。目标：

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta}[r_\phi(x, y)] - \beta\mathbb{E}_x\left[D_{\mathrm{KL}}\left(\pi_\theta(\cdot \mid x) \| \pi_{\mathrm{ref}}(\cdot \mid x)\right)\right].$$

KL 项的作用（**【讲义】Lec 14**）：防止为"刷奖励"输出不通顺乱码，也防止利用奖励模型漏洞（reward hacking）。

17.3 奖励模型：Bradley–Terry

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right), \qquad L(\phi) = -\mathbb{E}\left[\log\sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right].$$

直觉："更好的回答其奖励**指数级**更可能被选中"（与 Elo 同构）。讲义原话："这本质上就是逻辑回归"。实现要点：奖励头接在 LLM 最后 hidden state 上（常取最后 token）；**从 SFT 模型初始化**而非随机；**同一 prompt 内成对比较**才有意义；奖励做 mean/std 归一化；过拟合会导致策略钻空子（用 reward ensemble 或 KL 缓解）。

17.4 PPO–RLHF 的工程细节

四个模型：Actor（训）、Critic（训）、Reward（冻）、Reference（冻）。显存是主要瓶颈：4 个 7B 模型 ≈ 4×14GB（fp16）+ 激活；实践用 LoRA、offload、vLLM 分离推理。

逐 token 奖励："每个 token 付一点 KL 税，最后一个 token 拿总奖励"：

$$r_t = \begin{cases} -\beta\log\dfrac{\pi_\theta(y_t \mid \cdot)}{\pi_{\mathrm{ref}}(y_t \mid \cdot)}, & t < T \\ r_\phi(x, y) - \beta\log\dfrac{\pi_\theta(y_T \mid \cdot)}{\pi_{\mathrm{ref}}(y_T \mid \cdot)}, & t = T \end{cases}$$

```
for iteration:
    responses = [sample(pi_theta, p) for p in prompts]
    rewards   = [r_phi(p, y) for p, y in zip(prompts, responses)]
    logp, logp_ref = log_prob(pi_theta, responses), log_prob(pi_ref, responses)
    token_rewards = -beta * (logp - logp_ref); token_rewards[last] += rewards
    adv = gae(token_rewards, critic(prompts, responses), gamma=1.0, lam=0.95)
    ppo_update(pi_theta, critic, responses, adv, logp_old=logp)
```

17.5 GRPO：去掉 critic

动机：PPO–RLHF 的 critic 与 actor 同规模（显存翻倍），且 token 级值函数在稀疏奖励下很难学。**GRPO**（Shao et al. 2024, DeepSeekMath）用"同一 prompt 的一组回答的相对好坏"当优势，不需要 critic。

组归一化优势（HW4 PDF 的精确形式）：对 prompt x_i 采 G 个回答，

$$A_{i,j} = \frac{r_{i,j} - \mu_i}{\sigma_i + \epsilon}, \quad \mu_i = \frac{1}{G}\sum_j r_{i,j}, \quad \sigma_i = \sqrt{\frac{1}{G}\sum_j (r_{i,j} - \mu_i)^2}.$$

关键性质：$\sum_j A_{i,j} = 0$，**天生自带 baseline**；同一 $A_{i,j}$ 赋给该回答的**所有 token**（奖励只在最后给）。

GR-REINFORCE 目标（HW4）：

$$L_{\text{pg}}^{\text{GR-REINFORCE}} = -\frac{1}{N}\sum_{i=1}^{N} A_i \bar{\ell}_i(\theta), \qquad \bar{\ell}_i(\theta) = \frac{1}{T_i}\sum_{t=1}^{T_i} \log \pi_\theta(y_{i,t} \mid x_i, y_{i,<t}).$$

除以 T_i 是为避免长回答仅因 token 多而权重更大（HW4 明确说明）。

GRPO 目标（带 clip）：

$$\rho_{i,t} = \frac{\pi_\theta(y_{i,t} \mid \cdot)}{\pi_{\text{old}}(y_{i,t} \mid \cdot)}, \qquad L_{\text{pg}}^{\text{grpo}} = -\frac{1}{N}\sum_i \frac{1}{T_i}\sum_t \min\left(\rho_{i,t}A_i,\ \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)A_i\right).$$

与 PPO 的差别：PPO 用 GAE（token 级、有 critic），GRPO 用组归一化（序列级、无 critic）。

KL 的采样估计器（HW4 最精彩的细节）：精确 KL 要对整个词表求和，不可行。用单 token 估计器

$$\Delta(a) = \log \pi_{\text{ref}}(a \mid s) - \log \pi_\theta(a \mid s), \qquad \widehat{k}(a) = e^{\Delta(a)} - \Delta(a) - 1.$$

为何无偏：$a \sim \pi_\theta$ 时 $\mathbb{E}[e^{\Delta}] = \sum_a \pi_\theta \frac{\pi_{\text{ref}}}{\pi_\theta} = \sum_a \pi_{\text{ref}} = 1$，故 $\mathbb{E}[\widehat{k}] = -\mathbb{E}[\Delta] = \mathbf{KL}$。**为何不用 $-\Delta$**：$-\Delta$ 也无偏，但**单样本可能为负**（策略给该 token 的概率已低于参考时），逐 token 惩罚会把"更像参考"误判为奖励；而 $\widehat{k} = x - \log x - 1 \geq 0$（$x = e^{\Delta}$），**恒非负、仅在 $x = 1$ 时为 0**，是更自然的局部散度。

17.6 DPO：把 RL 变成分类【补充】

DPO（Rafailov et al. 2023）的起点是"KL 约束下最优策略有闭式解"：

$$\pi^\star(y \mid x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y \mid x)\exp\left(\frac{r(x,y)}{\beta}\right) \implies r(x,y) = \beta \log \frac{\pi^\star(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

代回 Bradley–Terry 似然，$Z(x)$ 在差值中消掉：

$$L_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right].$$

维度	PPO-RLHF	DPO
奖励模型	需要	**不需要**
在线采样	需要	不需要（离线偏好）
显存	4 个模型	2 个
稳定性	难调	简单
上限	通常更高（可在线探索）	受离线数据限制
失败	reward hacking	偏好数据覆盖不足时过度拟合

17.7 可验证奖励与推理 RL

HW4 的真实奖励设计（HW4 PDF）：`math_hard`（MATH level-5 子集，512 例固定 held-out 评估）——boxed 答案正确 **+1.0**、出现 `\boxed{` **+0.1**、宽松"最后数字"解析正确但 boxed 错 **+0.1**；`format_copy`（调试用）——整数正确 +1.0、含 `<answer>` +0.2、严格 XML +0.1。作业强调：**格式分是为了促进探索**（先学会"要输出 boxed"，再学会"boxed 里放对"），是 reward shaping 的正面案例。

奖励类型	定义	优点	缺点
结果奖励（verifier）	只看最终答案	便宜、难 hack	稀疏，长链推理难
过程奖励（process）	每步 CoT 打分	稠密	需 PRM，可能引入偏差

现代变体【补充】（arXiv 号见 **HW4 PDF** 的推荐阅读，已核对标题）：GRPO（2402.03300）、DrGRPO（2503.20783）、DAPO（2503.14476）、GSPO（2507.18071，序列级重要性比）。

17.8 失败模式

现象	根因	处理
输出变短/变乱	KL 系数太小	增大 β；检查 ref
奖励涨但人评降	reward hacking	reward ensemble、KL、早停
loss NaN	长序列 + fp16	bf16、梯度裁剪、降 lr
GRPO 组内奖励全同	任务太易/太难	调难度、增大 G、加格式分
熵塌缩	过度优化	熵正则、降 lr
评估虚高	训练集泄漏	严格 held-out

第 18 章 多任务、目标条件、分层与元学习

18.1 动机：从"一任务一策略"到通才

【讲义】Lec 24 用"通才模型"开篇：CV/NLP 已从"一任务一模型"转向"基础模型 + 微调/提示"，RL 也在走同一条路——在多任务上联合训练，**每个任务所需数据量大幅下降**，并为下游提供更好初始化。任务上下文（task context）可以是语言指令、目标状态、任务 ID、技能隐变量 z。

18.2 多任务 RL = 联合 MDP 上的单任务 RL

讲义的关键观察：多任务 RL **不是**新问题，它等价于在"联合 MDP"上做单任务 RL（第一个状态从 $p(\mathcal{T})$ 采任务，之后在 $\mathbf{MDP}_{\mathcal{T}}$ 里交互）。难在哪？

难点	说明
任务干扰	简单任务学得快，agent 忙于刷简单任务，难任务学不动（Schaul et al. "ray interference"）
奖励尺度不一致	不同任务回报量级不同，梯度被大任务主导
数据不平衡	简单任务数据多

解法：**课程**（从易到难或按学习进度采样）、**重标注**（复用经验）、**按任务归一化回报**、梯度平衡（PCGrad/GradNorm【补充】）。

18.3 目标条件 RL 与 HER

$$\pi(a \mid s, g), \qquad r(s, a, g) = \mathbb{1}\{\|s' - g\| \le \epsilon\} \text{ 或 } -\|s' - g\|.$$

优点（【讲义】Lec 24）：不需为每任务设计奖励；**零样本迁移**（新任务若是"到达某目标"，换 g 即可）。缺点：训练难（稀疏）、不是所有任务都是目标到达。

HER（Hindsight Experience Replay）：失败轨迹对原目标失败，但**对它实际到达的状态是成功的**：

```
episode = rollout(pi, goal=g)
for t, (s, a, s2) in enumerate(episode):
    buffer.add(s, a, s2, goal=g)          # 原目标
    buffer.add(s, a, s2, goal=s2)         # 事后重标注（"future" 策略）
```

把稀疏奖励变**稠密**（几乎每条轨迹都能提供"到达了某目标"的正样本）。**为什么必须 off-policy**（讲义提问）：重标注后的 (s, a, g') 不是当前策略在 g' 下采的，on-policy 算法不能用。

目标条件的几何：若 $V(s, g)$ 是"到 g 的负距离"，则满足度量性质（三角不等式），可用于规划 $g^\star = \arg\min_g V(s, g) + V(g, g_{\text{target}})$，或作辅助损失 $V(s, g) \ge V(s, g') + V(g', g)$。

18.4 后继特征

若奖励可写成特征线性组合 $r(s) = \phi(s)^\top w$，则

$$V^\pi(s) = \mathbb{E}\left[\sum_t \gamma^t \phi(s_t)\right]^\top w = \psi^\pi(s)^\top w.$$

ψ^π 只依赖动力学与策略，**换奖励 = 换 w，不用重学 ψ**。用途：快速适应新奖励、多任务共享、理论（后继表示是"模型的压缩"，Dayan 1993）。局限：需要"奖励是特征线性函数"；连续状态下 ψ 是密度而非计数，更难估（C-Learning 用分类器绕过）。

18.5 分层 RL

options 框架（【讲义】Lec 24）：$(\mathcal{I}, \pi, \beta)$ = 初始集 + 内部策略 + 终止条件；高层选 option，低层执行到终止。

好处	代价
时间抽象（长程信用分配变短）	option 从哪来?
可复用子技能	端到端联合学习会"退化成平坦策略"
探索更结构化	终止条件难学

现代做法：HIRO/HRAC（高层给目标、低层做目标条件 RL）、层次化目标（高层每 c 步给子目标）、语言分层（ArCHer：高层 LLM 规划、低层 RL 执行，**【讲义】Lec 24** 引 Zhou et al.）。

18.6 元学习（meta-RL）

$$\min_{\theta} \mathbb{E}_{\mathcal{T}\sim p(\mathcal{T})}\left[\mathcal{L}\left(\text{adapt}(\theta, \mathcal{D}_{\mathcal{T}}^{\text{train}}), \mathcal{D}_{\mathcal{T}}^{\text{test}}\right)\right].$$

循环策略（**【讲义】Lec 24**）：用 RNN/Transformer 当策略，**meta-episode 内不重置 hidden state**，奖励序列作为输入。因为优化的是整个 meta-episode 的总回报，前期探索的代价被后期收益补偿，网络**自动学会"先探索、再利用"**。这其实就是 LLM **in-context learning** 的 RL 版本（讲义明确点了这个类比）。

meta-RL = POMDP（任务身份是隐状态 z），故所有 POMDP 工具可用：循环策略（隐状态编码任务）、变分推断（PEARL 用 $q(z \mid \text{context})$）、梯度-based（MAML 用几步梯度适应）。

18.7 失败模式

现象	根因	处理
简单任务霸占训练	ray interference	课程、任务重采样、梯度平衡
只学会近目标	目标采样分布太窄	HER、Skew-Fit
分层退化为平坦	端到端联合训练	分阶段训练、信息瓶颈
meta-RL 不探索	meta-episode 太短	加长、加熵
多任务负迁移	任务冲突	任务 embedding、参数隔离

第 19 章 开放问题：RL 还缺什么

19.1 三个视角

【讲义】Lec 25：

视角	主张	现状
工程工具	"任何你能仿真的东西，你都能控制"	工业界主流（AlphaGo、LLM RL、sim-to-real）

视角	主张	现状
真实世界	应在"规则未知、必须活下去"的环境工作	仍是前沿
通用学习	"廉价历史数据 + 少量监督"的通才范式	早期

easy vs hard universe（最值得记住的一张表）：

维度	简单宇宙	困难宇宙
成功定义	高奖励（最优控制）	"活下来"（足够好的控制）
世界	封闭、规则已知	开放、规则靠数据发现
仿真	有	没有（规则未知）
核心问题	**能不能优化好**	**能不能泛化与适应**

讲义的论断：**RL 在原理上应擅长"困难宇宙"，但研究几乎都在"简单宇宙"里做**——这是 Moravec 悖论在 RL 上的投影。

19.2 现实世界的五个问题

【讲义】Lec 25：① 怎么告诉 agent 我们想要什么（奖励设计/对齐）；② 怎么在持续变化的环境里完全自主学习（continual learning）；③ 环境变化时怎么保持鲁棒；④ 怎么正确使用"经验 + 先验数据"；⑤ 怎么用先验经验加速探索。

19.3 与 LLM 的交汇

【讲义】Lec 25 给了一个"模型-based RL 用于对话"的例子（Hong, Levine, Dragan 2023）：用 LLM 生成"想象中的对话"（synthetic data）→ 用"人类模拟器"评估 → RL 在这个想象环境里优化对话策略 → 学到"该问什么澄清问题"。关键洞察：**RL + LLM 适合"LLM 能模拟但做不好"的任务**（如主动提问）——它知道人类会怎么答，但不擅长"为获取信息而提问"。

19.4 我的判断（明确标注为推断）

- **短期（1–2 年）**：最大落地仍是"后训练"（RLHF/RLVR）与机器人 sim-to-real，而非通用 agent。
- **中期**：离线 RL + 世界模型（"在想象中做 offline RL"）可能是数据效率的突破口。
- **长期**：瓶颈不是算法而是**评估**——我们缺少衡量"开放世界适应能力"的 benchmark。

讲义的四条研究建议对学习者同样适用：**选对问题**（"这有没有可能解决一个重要问题"）；**敢于改问题陈述**（很多挑战不会靠刷现有 benchmark 解决）；**应用很重要**（真实约束会教你别处学不到的东西）；**"Think big and start small"**。

第 20 章 与已有笔记的衔接

20.1 CS234：理论骨架 ↔ 工程细节

在 CS234 学到的	在 CS285 里如何被"工程化"
贝尔曼算子是 γ-压缩 → 值迭代收敛	一旦加函数逼近，投影破坏压缩性 → 需要 target network + replay（7.3–7.4）
表格 Q-learning 的 GLIE 收敛条件	深度 Q 没有这种保证 → 用 Double、Huber、多 seed 兜底（8.1）
策略梯度 = $\mathbb{E}[\nabla \log \pi A]$	用 GAE 把 A 的偏差-方差做成可调旋钮（6.2）
保守策略迭代 / 单调改进	TRPO 把"改进下界"变成 KL 硬约束（11.3）
UCB 的 $O(\log T)$ 遗憾	高维状态无法计数 → 伪计数、RND（15.3）
BC 的 $O(T^2\epsilon)$、DAgger 归约	多模态动作 + 扩散/flow + action chunking（2.4）
Bradley-Terry、DPO 推导	PPO-RLHF 的逐 token KL、GRPO 的组归一化与 KL 估计器（17.4–17.5）

建议读法：先读 CS234 的第 4–8 讲（值方法、PG、模仿、RLHF），再读本笔记对应章节，你会发现"公式一模一样，但工程约束完全不同"。

20.2 CS229：监督学习是 RL 的子程序

- BC = 最大似然（2.2）；奖励模型 = 逻辑回归（17.3）；VAE/ELBO = 变分推断（12.2）。
- CS229 的"偏差-方差权衡"在这里变成"自举深度"的权衡：λ（GAE）、n（n-step）、H（MBPO rollout 长度）都是同一个旋钮的不同名字。
- CS229 的"生成模型 vs 判别模型"对应 IRL 的两种实现：显式奖励（生成）vs 判别器（GAIL，3.2）。

20.3 CS336：LLM 训练栈 ↔ RL 后训练

CS336 的主题	与 CS285 的交集
tokenizer / 模型并行 / 显存优化	PPO-RLHF 的 4 模型显存问题（17.4）；GRPO 去 critic 的动机
SFT / 指令微调	后训练的第一段（17.1）；"预训练 broad + 后训练 narrow"（2.5）
采样（temperature/top-p）	RL 的 rollout 采样；KL 正则防"跑飞"（17.2）
评估与去污染	RLVR 的 verifier 设计、held-out 评估（17.7）

一句话：**CS336 教你怎么把模型训出来，CS285 教你训出来之后怎么用奖励把它"掰"向人类想要的行为。**

20.4 CS231n：视觉表示 ↔ 策略的输入

- 视觉 encoder（ResNet/ViT）是 BC 与 RL 的共同前端（2.2 的 encoder）。
- 数据增广在 BC 里不是"正则"，而是**扩展状态分布**（2.3 的左右摄像头）。
- 世界模型的"重建 vs 不重建"之争（13.6），本质是 CS231n 里"生成式 vs 判别式表示"的老问题。
- 视觉 RL 的样本效率极低，是 model-based RL 与表征学习的主战场（Dreamer、TD-MPC2）。

第 21 章 复习清单与自测题

21.1 复习清单（按"能否不看笔记讲 3 分钟"自评）

#	主题	应能回答
1	算法解剖	三步骨架是什么？四类算法各在哪一步不同？
2	BC	为什么误差是 $O(\epsilon T^2)$ 而不是 $O(\epsilon T)$？
3	DAgger	为什么"聚合"不可省？为什么它需要交互式专家？
4	似然比	写出 $\nabla_\theta J$ 并说明环境项为何消失
5	baseline	证明减 baseline 无偏；最优 baseline 是什么
6	GAE	写出递推式；$\lambda=0$ 与 $\lambda=1$ 各是什么
7	DQN	为什么需要 target network？Double Q 修正了什么？
8	过估计	用 $\mathbb{E}[\max] \geq \max \mathbb{E}$ 解释来源
9	连续动作	DDPG 的确定性策略梯度怎么来？TD3 三个补丁？
10	SAC	软备份公式；温度如何自动调；tanh 的 log-det
11	PPO	clip 的四种情形；为什么它是 KL 版的近似
12	TRPO	单调改进下界；自然梯度为何用 F^{-1}
13	控制即推断	软值迭代 = log-sum-exp；熵从哪来
14	MBRL	模型误差为何被放大；MBPO 的短 rollout 为何有效
15	离线 RL	反事实查询；策略约束/保守/避 OOD 三条路线
16	IQL	expectile 如何近似 max；为何不查 OOD
17	探索	伪计数公式；RND 为何用冻结随机网络
18	理论	$(1-\gamma)^{-2}$ 从哪来；为何说"没有保证"
19	RLHF	Bradley-Terry；逐 token KL；GRPO 组归一化

#	主题	应能回答
20	GRPO/KL	为何用 $e^{\Delta}-\Delta-1$ 而不是 $-\Delta$

21.2 自测题（20 题）

1. 写出策略梯度的似然比形式，并说明为什么 $\log p(s_{t+1}\mid s_t,a_t)$ 求导后消失。
2. reward-to-go 与全轨迹回报在期望上相等，为什么方差更小？给出关键恒等式。
3. 若 batch size 为 1，优势归一化后会发生什么？这说明它是有偏的还是无偏的？
4. GAE 中 $\lambda=0$ 与 $\lambda=1$ 分别对应什么估计器？偏差与方差如何变化？
5. 为什么 DQN 的 target 需要"冻结"？不冻结会出现什么现象？
6. Double DQN 用在线网络选动作、目标网络估值，这为什么不引入额外偏差？
7. 用一句话解释 $\mathbb{E}[\max_a \hat{Q}]\geq\max_a\mathbb{E}[\hat{Q}]$ 与 Q 值高估的关系。
8. DDPG 的 actor 梯度与 SAC 的重参数化梯度有何本质区别？
9. TD3 的 target policy smoothing 解决什么问题？为什么"平滑"能起作用？
10. SAC 的 α 自动调节：写出 L_α 并说明 $\bar{\mathcal{H}}=-|\mathcal{A}|$ 的含义。
11. PPO-clip 中，当 $\hat{A}_t>0$ 且 $\rho_t>1+\epsilon$ 时梯度会发生什么？
12. 为什么说 PPO 的 clip 是 KL 惩罚版的"启发式近似"？
13. TRPO 的单调改进下界中，惩罚系数为什么很大？为什么改用硬约束？
14. 自然梯度 $\theta\leftarrow\theta+\alpha F^{-1}g$ 中 F 是什么？为什么不用普通梯度？
15. 控制即推断里，"乐观问题"是什么？变分修正如何解决它？
16. 模型-based RL 中，为什么"模型更准"不等于"策略更好"？举一个失败机制。
17. MBPO 为什么要用"短 rollout + 真实起点"？两个设计各解决什么问题？
18. 离线 RL 中，CQL 与 IQL 处理 OOD 的思路有何根本不同？
19. RND 为什么用"冻结的随机网络"而不是"可学习的前向模型"？
20. GRPO 的 KL 估计器为什么必须非负？给出一个会因 $-\Delta$ 出错的场景。

21.3 答案

1. $\nabla_\theta J=\mathbb{E}[\sum_t\nabla\log\pi_\theta(a_t\mid s_t)\sum_{t'}\gamma^{t'}r_{t'}]$。因为 $\log p_\theta(\tau)$ 中环境项 $\log p(s_{t+1}\mid s_t,a_t)$ 不含 θ，求导为零。
2. 关键恒等式 $\mathbb{E}_{a_t}[\nabla_\theta\log\pi_\theta(a_t\mid s_t)]=0$，故 t 之前的历史奖励乘上该梯度后期望为 0，可从权重中删去；删去随机项后方差下降。
3. 归一化后优势恒为 0，梯度为 0。说明归一化**有偏**（依赖 batch 统计量），但 batch 大时偏差可忽略。
4. $\lambda=0$ → 单步 TD δ_t（偏差最大、方差最小）；$\lambda=1$ → MC 减 baseline（无偏、方差最大）。
5. 因为 target 依赖被优化参数，不冻结则"追自己的尾巴"，回归目标不断移动，无稳定解，Q 值发散或震荡。

6. 它只改变"选哪个动作"，不改变"估值的网络"；选择与评估用不同噪声源，故 $\mathbb{E}[Q_{\phi'}(s', \arg\max Q_{\phi})]$ 不会系统性地取到噪声上界，减小过估计而不引入额外偏差项（严格说仍有来自 ϕ' 滞后的偏差）。
7. $\max$ 是凸函数，Jensen 不等式给出"噪声的 max 的期望 ⩾ 期望的 max"；TD 目标用 $\max$ 就会系统性取到偏高值，并经自举累积。
8. DDPG 对**确定性** actor 用链式法则 $\nabla_a Q \cdot \nabla_\theta \mu$（路径导数，但策略无随机性，探索靠外挂噪声）；SAC 对**随机**策略用重参数化 $a = \mu + \sigma\epsilon$，梯度穿过采样，且熵项使策略保持随机。
9. 解决 actor 利用 Q 函数在"动作维度上的尖峰/错误高估"（adversarial 动作）。平滑使 target 在动作邻域内取期望，抹平尖峰。
10. $L_\alpha = \mathbb{E}[-\alpha(\log \pi_\theta(a \mid s) + \bar{\mathcal{H}})]$；$\bar{\mathcal{H}} = -|\mathcal{A}|$ 表示"目标熵 = 负的动作维数"，即每个动作维度约 1 nat 的熵，是常用的启发式。
11. $\min$ 选中被裁剪的那一项（常数），**梯度为 0**——不再奖励"继续提高该动作概率"。
12. 把 KL 惩罚的软约束换成对 ρ 的硬裁剪，只保留一阶信息、无二阶曲率，故只是近似；但实践中更稳更好调。
13. 系数 $\frac{2\epsilon\gamma}{(1-\gamma)^2}$ 含 $\epsilon = \max_s |\mathbb{E}A|$，难以准确估计且通常过大，会导致步长极小；改用硬约束 $\mathbb{E}[\mathrm{KL}] \le \delta$ 可直接控制步长。
14. $F = \mathbb{E}[\nabla \log \pi_\theta \nabla \log \pi_\theta^\top]$ 是 Fisher 信息矩阵（KL 的二阶近似）。参数空间欧氏梯度不考虑"分布距离"，会在大曲率方向走太远；F^{-1} 把梯度拉回"分布空间的最速方向"。
15. 乐观问题指反向消息会偏好"靠运气到达高价值状态"的转移（会去撞随机传送门）。变分修正把转移项纳入下界，得到"奖励 + 熵 $-\log p(s' \mid s, a)$"，即 maximum entropy RL。
16. 策略优化会主动寻找模型的高误差区域（那里"看起来"回报高）；模型整体更准不代表在**策略访问的**区域更准。失败机制：模型在稀有状态高估奖励 → 策略被吸过去 → 真实环境崩。
17. 短 rollout 解决"误差随 H 累积"；真实起点解决"起点分布偏移、覆盖不足"。二者共同把模型误差限制在 H 步内，长程信用分配交给 Q 自举。
18. CQL **修正 Q**（对 OOD 动作压低、数据动作抬高），策略可以自由选动作；IQL **避免查询 OOD**（V 只用数据动作做 expectile 回归，策略只做加权 BC）。
19. 可学习前向模型的误差会被环境随机性（aleatoric）污染，导致"噪声电视"问题；冻结随机网络是**确定性**目标，同一状态永远给同一目标，误差只反映"是否见过"。
20. 因为 KL 本身非负，逐 token 惩罚也应非负；若用 $-\Delta$，当策略给该 token 的概率已低于参考时 $\Delta > 0$、$-\Delta < 0$，会把"已经更保守"误判为奖励，导致估计器鼓励偏离参考。

第 22 章 参考来源

> 全部链接访问日期：**2026–11**。课程材料会随学期更新，具体日期/分数/作业门槛**以课程当学期主页为准**。

22.1 课程与作业（一手来源）

内容	URL
课程主页（Spring 2026 日程、讲次、作业、项目）	https://rail.eecs.berkeley.edu/deeprlcourse/
Syllabus（先修、评分、late policy、协作规则）	https://rail.eecs.berkeley.edu/deeprlcourse/syllabus
Resources（往届课程、教材列表）	https://rail.eecs.berkeley.edu/deeprlcourse/resources
讲义 1–25（`lec-N.pdf`）	`https://rail.eecs.berkeley.edu/deeprlcourse/static/slides/lec-1.pdf` … `lec-25.pdf`
讨论课讲义（section-1 … section-9）	`https://rail.eecs.berkeley.edu/deeprlcourse/static/sections/section-1.pdf` 等
HW1 Imitation Learning	https://rail.eecs.berkeley.edu/deeprlcourse/static/homeworks/hw1.pdf
HW2 Policy Gradients	https://rail.eecs.berkeley.edu/deeprlcourse/static/homeworks/hw2.pdf
HW3 Q-Learning and Actor Critic	https://rail.eecs.berkeley.edu/deeprlcourse/static/homeworks/hw3.pdf
HW4 LLM RL	https://rail.eecs.berkeley.edu/deeprlcourse/static/homeworks/hw4.pdf
HW5 Offline RL	https://rail.eecs.berkeley.edu/deeprlcourse/static/homeworks/hw5.pdf
起始代码仓库（Spring 2026）	https://github.com/berkeleydeeprlcourse/homework_spring2026
默认期末项目：Offline-to-Online RL	https://rail.eecs.berkeley.edu/deeprlcourse/static/misc/offline_to_online_rl_default_final_project.pdf
默认期末项目：LLM RL	https://rail.eecs.berkeley.edu/deeprlcourse/static/misc/llm_rl_default_final_project.pdf
期末项目说明	https://rail.eecs.berkeley.edu/deeprlcourse/static/misc/final_project_outline.pdf

22.2 教材

教材	URL
Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed., MIT Press 2018)，免费 PDF	http://incompleteideas.net/book/the-book-2nd.html
课程 Syllabus 推荐的 RL/MDP 入门（CS188 EdX）	见 Syllabus 的 Prerequisites 一节

22.3 论文（按章节）

以下 arXiv 编号均已核对标题（通过 arXiv API 查询）：

章节	论文	arXiv
3	Ross et al., *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning* (DAgger)	1011.0686
3	Finn et al., *Guided Cost Learning*	1603.00448
3	Ho & Ermon, *Generative Adversarial Imitation Learning*	1606.03476
5	Schulman et al., *High-Dimensional Continuous Control Using GAE*	1506.02438
7	Mnih et al., *Playing Atari with Deep Reinforcement Learning* (DQN)	1312.5602
8	van Hasselt et al., *Deep RL with Double Q-learning*	1509.06461
8	Wang et al., *Dueling Network Architectures*	1511.06581
8	Schaul et al., *Prioritized Experience Replay*	1511.05952
8	Hessel et al., *Rainbow*	1710.02298
9	Lillicrap et al., *Continuous control with deep RL* (DDPG)	1509.02971
9	Fujimoto et al., *Addressing Function Approximation Error in Actor-Critic* (TD3)	1802.09477
9	Haarnoja et al., *Soft Actor-Critic*	1801.01290
9	Haarnoja et al., *Soft Actor-Critic Algorithms and Applications*（自动温度）	1812.05905
10	Schulman et al., *Proximal Policy Optimization Algorithms*	1707.06347
11	Schulman et al., *Trust Region Policy Optimization*	1502.05477
13	Janner et al., *When to Trust Your Model* (MBPO)	1906.08253
14	Kumar et al., *Conservative Q-Learning* (CQL)	2006.04779
14	Kostrikov et al., *Offline RL with Implicit Q-Learning* (IQL)	2110.06169
14	Fujimoto et al., *Off-Policy Deep RL without Exploration* (BCQ)【补充】	1812.02900
14	Chen et al., *Decision Transformer*【补充】	2106.01345
15	Bellemare et al., *Unifying Count-Based Exploration and Intrinsic Motivation*（伪计数）	1606.01868
15	Burda et al., *Exploration by Random Network Distillation* (RND)	1810.12894
17	Shao et al., *DeepSeekMath*（GRPO）	2402.03300
17	*Understanding R1-Zero-Like Training: A Critical Perspective*（DrGRPO）	2503.20783
17	*DAPO: An Open-Source LLM RL System at Scale*	2503.14476
17	*Group Sequence Policy Optimization*（GSPO）	2507.18071

22.4 说明与未覆盖项

- **未逐页精读的材料**：25 份讲义 PDF 全部下载并转为文本通读；5 份作业 PDF 全部通读；仓库 README、`model.py` / `train.py` / `pg_agent.py` / `dqn_agent.py` / `sacbc_agent.py` / `iql_agent.py` / `fql_agent.py` 的 TODO 结构已核对。仓库其余源码（如 `rollout/hf_sampler.py`、`tasks/math_hard.py` 全文）只读了 README 中的描述，未逐行阅读。
- **无法核对的内容**：讲义 PDF 中的**公式与图**多为图片，`pdftotext` 只能提取文字，因此公式依据的是我已有的领域知识 + 作业 PDF 中的排版公式；若与讲义图上的写法有差异，以讲义原图为准。
- **学期相关**：本笔记的评分、周次、作业门槛均来自 **Spring 2026** 版本；往届（Fall 2017–2023）作业编号与主题不同（例如旧版 HW4 是 SAC 而非 LLM RL），请勿混用。
- **未做的事**：未做任何 git 操作；未生成 PDF。